INSTRUMENTATION ENGINEERING LABORATORY 2

01068102

LAB 3 : AppDesigner and Stand-alone

Instrumentation and Control Engineering,
School of Engineering,
King Mongkut's Institute of Technology Ladkrabang

Assoc. Prof. Dr. Arjin Numsomran

วัตถุประสงค์

SCHOOL OF ENGINEERING KMITL

1. นักศึกษาเข้าใจหลักการใช้งาน MATLAB/AppDesigner
2. นักศึกษาสามารถออกแบบ MATLAB/AppDesigner สำหรับระบบควบคุมได้
3. นักศึกษาได้เรียนรู้และวิเคราะห์กระบวนการชนิดต่าง ๆ
4. นักศึกษาสมารถใช้เครื่องมือ MATLAB/SIMULINK เชื่อมต่อกับอุปกรณ์ผ่าน ไมโครคอนโทลเลอร์ (HIL, PIL, Stand-alone)
5. นักศึกษาสามารถออกแบบตัวควบคุม PID สำหรับการควบคุมกระบวนการได้จริง

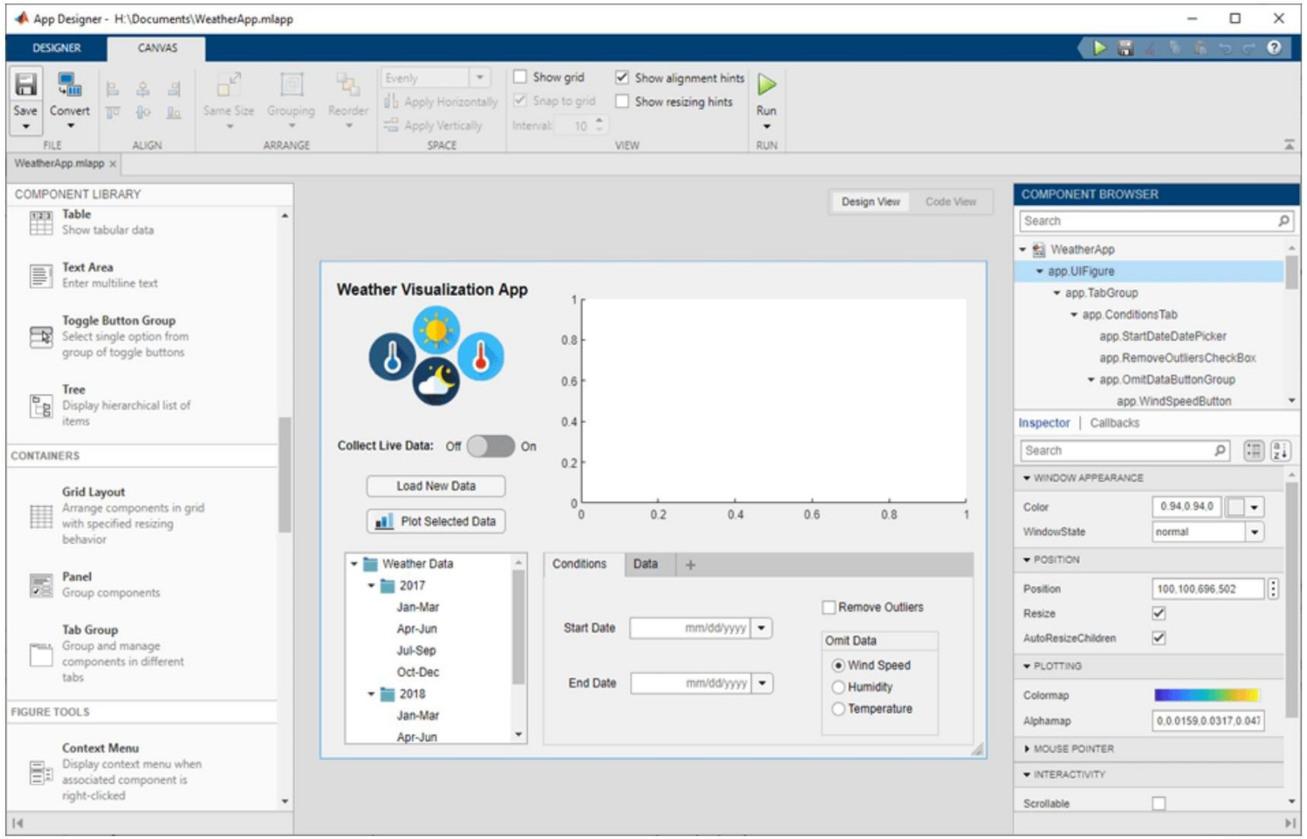
หัวข้อที่เกี่ยวข้อง

SCHOOL OF ENGINEERING KMITL

1. พื้นฐานการใช้งานเครื่องมือ AppDesigner
2. ตัวอย่าง Arduino LED Control over Serial
3. การติดตั้ง Simulink Support Package for Arduino Hardware
4. การทดลองควบคุมระบบไฟฟ้า (Electrical system)
5. การทดลองควบคุมอุณภูมิ (Lightbulb)
6. การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)
7. PID Stand-Alone with Arduino Uno R3
8. คำถามท้ายการทดลอง

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- App Designer
  - เป็นอีกความสามารถหนึ่งของ MATLAB ที่นำมาพัฒนา UI โต้ตอบกับผู้ใช้งานซึ่งเปิดตัวใน R2016a สามารถ สร้าง Layout จาก Components ต่าง ๆ เพื่องานประยุกต์ที่ต้องการได้หลากหลาย



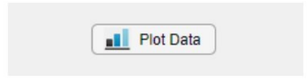
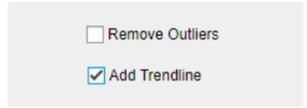
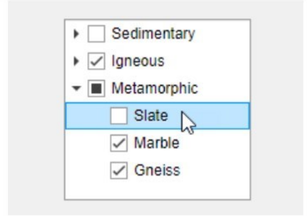
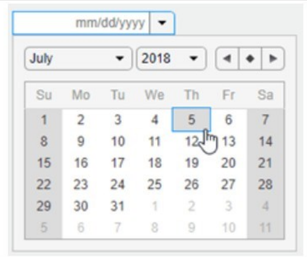
Ref: https://www.mathworks.com/help/matlab/creating_guis/ways-to-build-matlab-guis.html

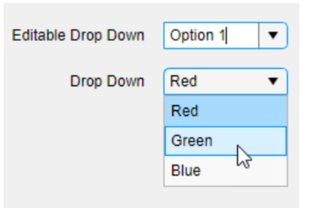
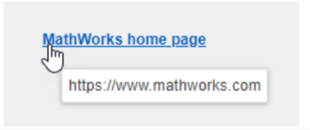
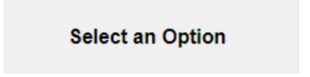
3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

SCHOOL OF ENGINEERING KMITL

- Components

Common Components

Component Information	Example
Button Properties	
CheckBox Properties	
CheckBoxTree Properties TreeNode Properties	
DatePicker Properties	

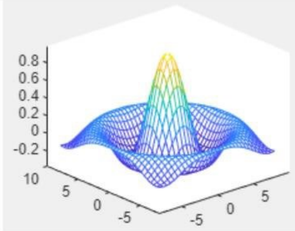
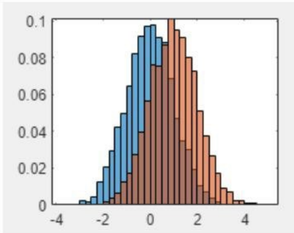
DropDown Properties	
NumericEditField Properties	
EditField Properties	
Hyperlink Properties	
Image Properties	
Label Properties	
ListBox Properties	

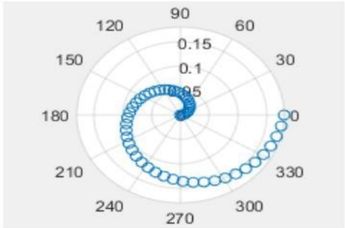
Ref: https://www.mathworks.com/help/matlab/creating_guis/ways-to-build-matlab-guis.html

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Axes

Axes

Axes Information	Example
UIAxes Properties	
Axes Properties This object can be added programmatically only.	
GeographicAxes Properties This object can be added programmatically only.	

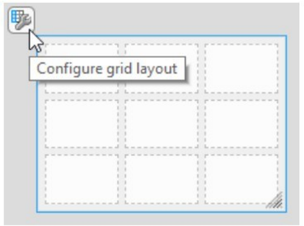
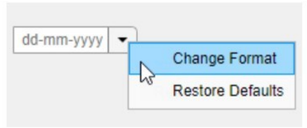
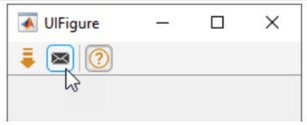
PolarAxes Properties This object can be added programmatically only.	

Ref: https://www.mathworks.com/help/matlab/creating_guis/ways-to-build-matlab-guis.html

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Containers and Figure Tool

Containers and Figure Tools

Container Information	Example
GridLayout Properties	
Panel Properties	
TabGroup Properties Tab Properties	
Menu Properties	
ContextMenu Properties	
Toolbar Properties PushTool Properties ToggleTool Properties	

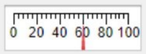
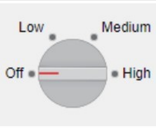
Ref: https://www.mathworks.com/help/matlab/creating_guis/ways-to-build-matlab-guis.html

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

SCHOOL OF ENGINEERING KMITL

- Instrumentation Components

Instrumentation

Component Information	Example
Gauge Properties	
NinetyDegreeGauge Properties	
LinearGauge Properties	
SemicircularGauge Properties	
Knob Properties	
DiscreteKnob Properties	

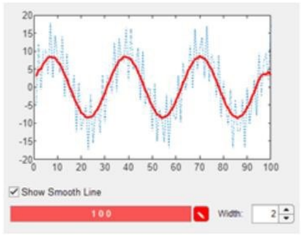
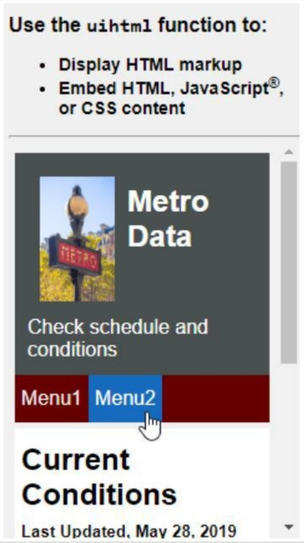
Lamp Properties	
Switch Properties	
RockerSwitch Properties	
ToggleSwitch Properties	

Ref: https://www.mathworks.com/help/matlab/creating_guis/ways-to-build-matlab-guis.html

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Extensible Components

Extensible Components

Component Information	Example
`matlab.ui.componentcontainer.ComponentContainer` Class `matlab.graphics.chartcontainer.ChartContainer` Class	
HTML Properties	

Ref: https://www.mathworks.com/help/matlab/creating_guis/ways-to-build-matlab-guis.html

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

SCHOOL OF ENGINEERING KMITL

- Toolbox Components

Toolbox Components

Apps created in App Designer or with the `uifigure` function support Aerospace Toolbox components. For more information, see Flight Instruments (Aerospace Toolbox). To use toolbox components, a valid license and installation of the associated toolbox is required.

Flight Instruments

R2021a

Display measurements through standard cockpit instruments

To display flight status information for your application, such as altitude and heading, use the cockpit instrument functions and their properties or gauges in MATLAB® App Designer. For example, you can use these functions and properties or gauges to replicate the look and feel of a standard cockpit. Create these standard cockpit instruments. Aerospace Toolbox creates the cockpit instruments as components. Use the associated properties to control the appearance and behavior of these cockpit instrument components.

The functions work only with figures created with the `uifigure` function. Apps created with GUIDE or the `figure` function do not support flight instrument components. With an Aerospace Toolbox license, flight instruments gauges are also available through MATLAB App Designer.

Functions

`uiaeroairspeed`	Create airspeed indicator component
`uiaeroaltimeter`	Create altimeter component
`uiaeroclimb`	Create climb rate indicator component
`uiaeroegt`	Create exhaust gas temperature (EGT) indicator component
`uiaeroheading`	Create heading indicator component
`uiaerohorizon`	Create artificial horizon component
`uiaerorpm`	Create revolutions per minute (RPM) indicator component
`uiaeroturn`	Create turn coordinator component

Properties

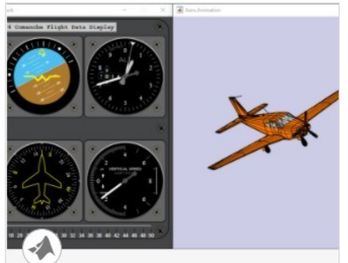
AirspeedIndicator Properties	Control airspeed indicator appearance and behavior
Altimeter Properties	Control altimeter appearance and behavior
ArtificialHorizon Properties	Control artificial horizon appearance and behavior
ClimbIndicator Properties	Control climb indicator appearance and behavior
EGTIndicator Properties	Control exhaust gas temperature (EGT) indicator appearance and behavior
HeadingIndicator Properties	Control heading indicator appearance and behavior
RPMIndicator Properties	Control revolutions per minute (RPM) indicator appearance and behavior
TurnCoordinator Properties	Control turn coordinator appearance and behavior

Featured Examples



Display Flight Trajectory Data Using Flight Instruments and Flight…

Visualize flight trajectories in a UI figure window using flight instrument components. In this example, you will create and configure standard



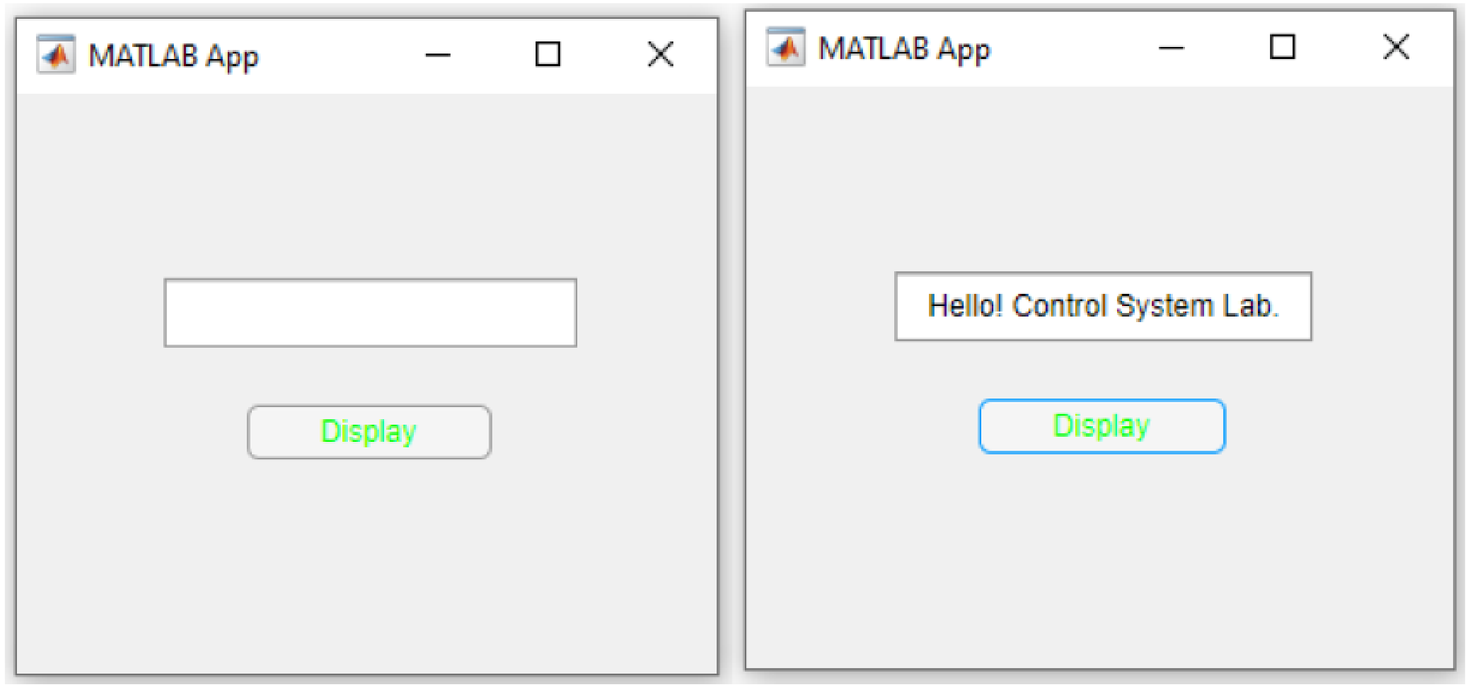
Aerospace Flight Instruments in App Designer

This app shows how to display flight status information with standard cockpit instrumentation using Aerospace Toolbox flight instruments

Ref: https://www.mathworks.com/help/matlab/creating_guis/ways-to-build-matlab-guis.html

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

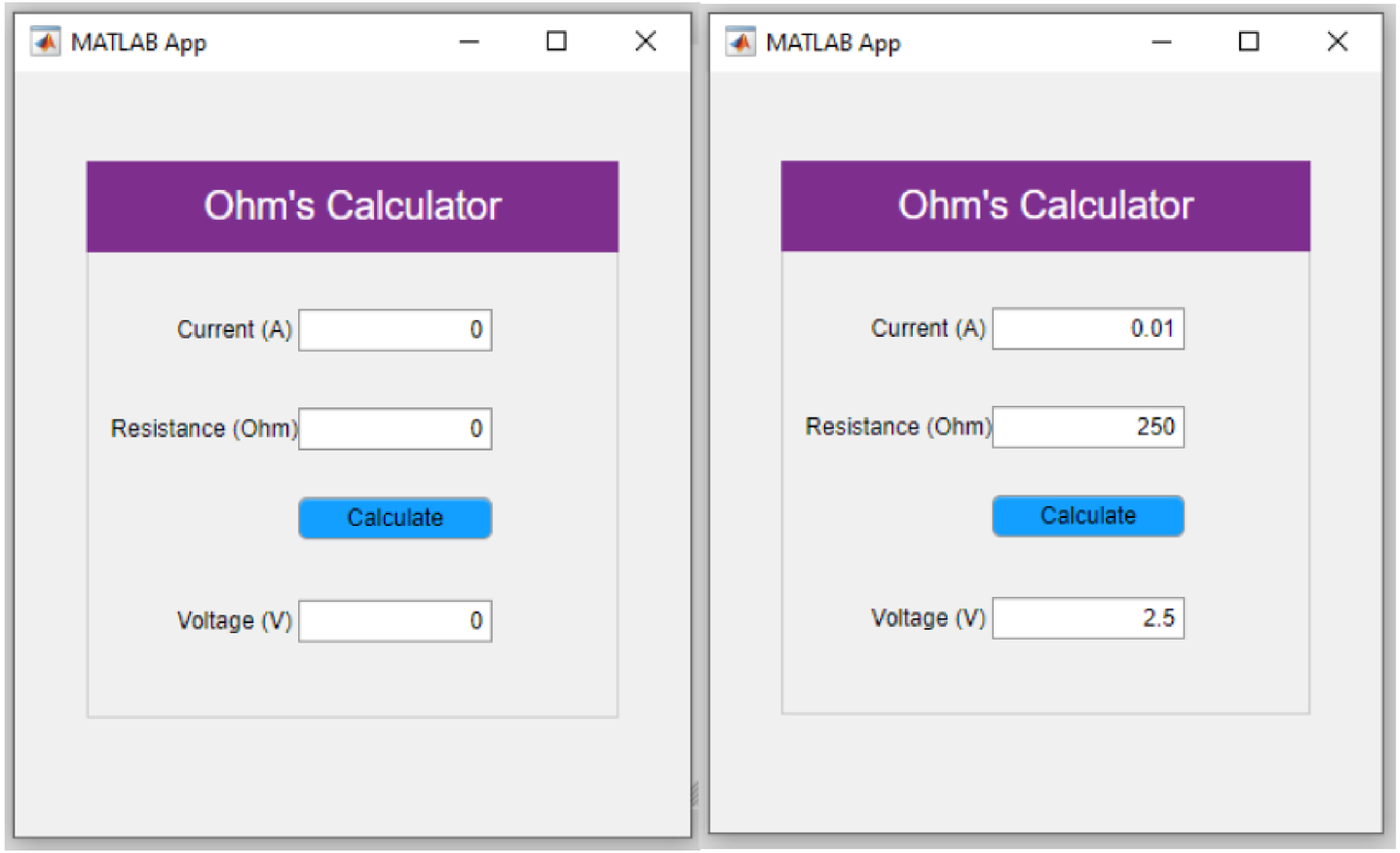
- Hello World




```
app.EditField.Value = 'Hello! Control System Lab.';
```

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

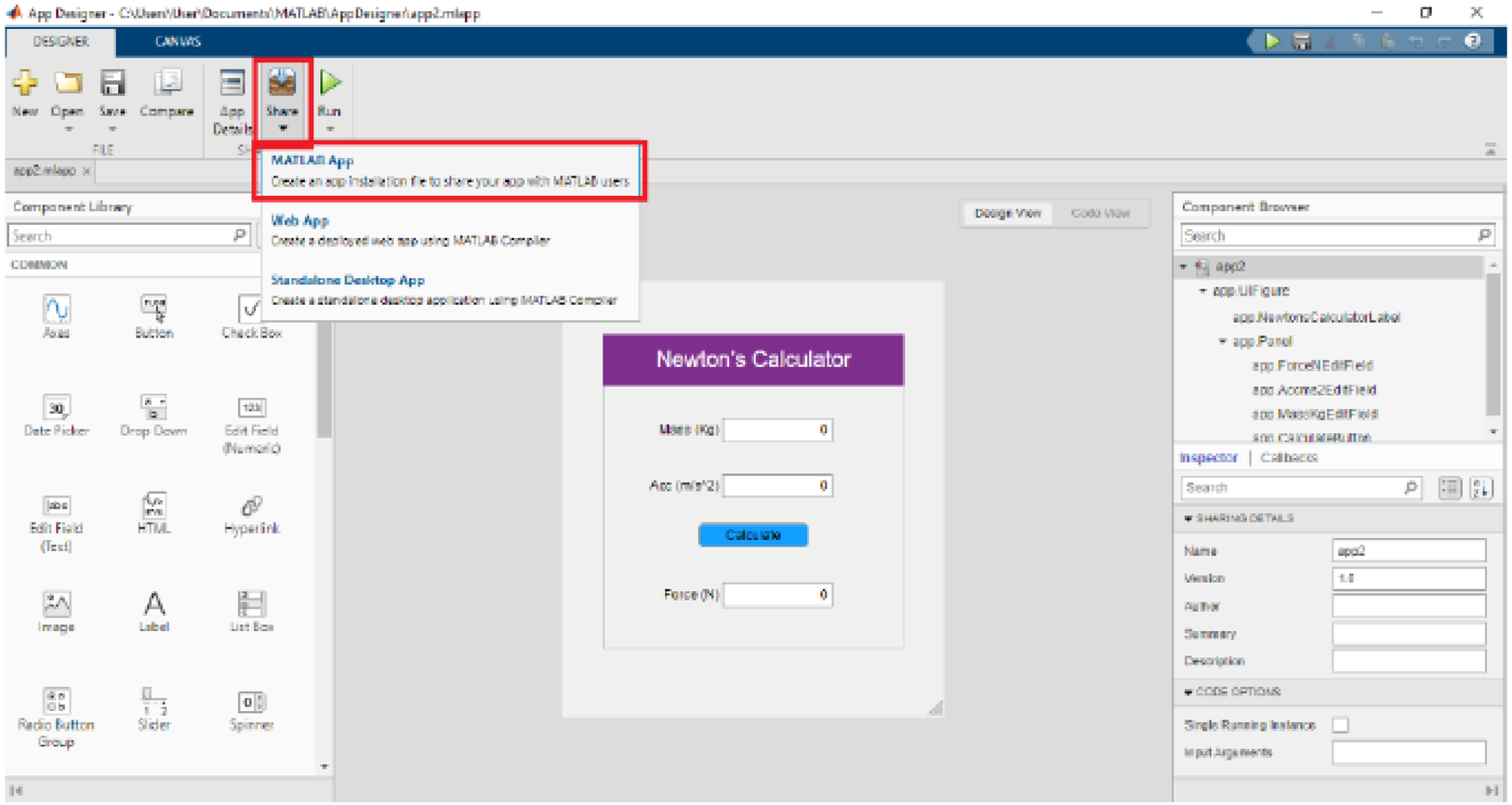
- Calculator




```
I = app.CurrentAEditField.Value;
R = app.ResistanceOhmEditField.Value;
% V = IR
V = I*R;
app.VoltageVEditField.Value = V;
```

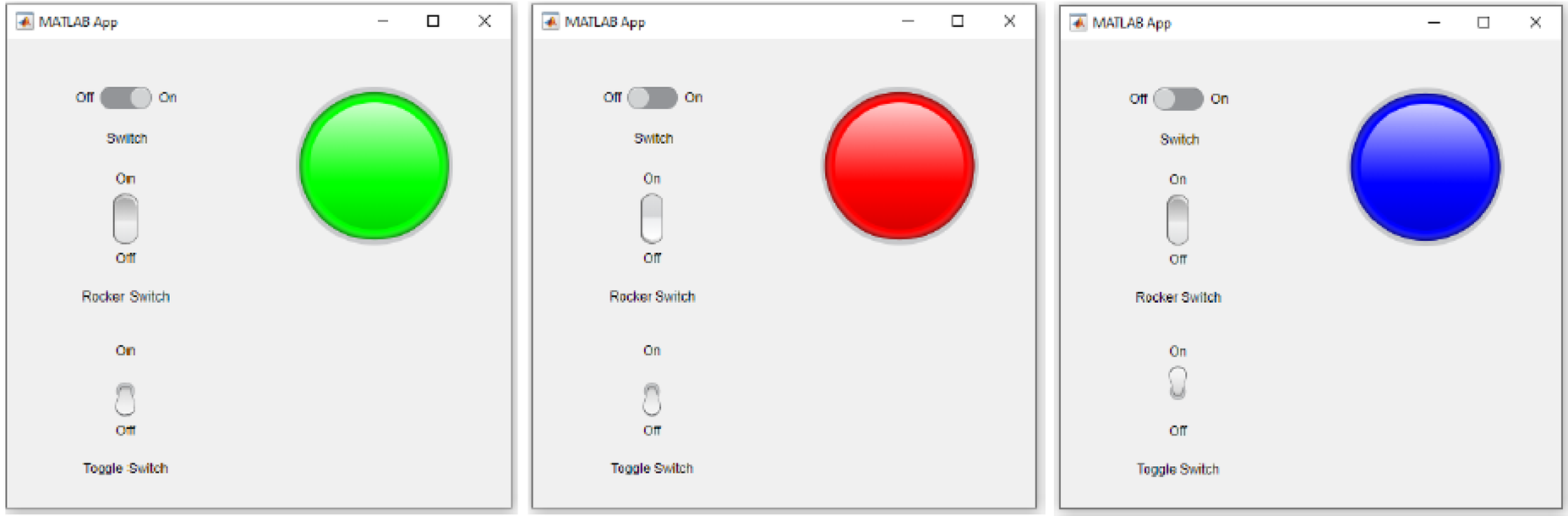
3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Compile/Package an App



3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Lamp Indicator/Switch/Knob /Gauge



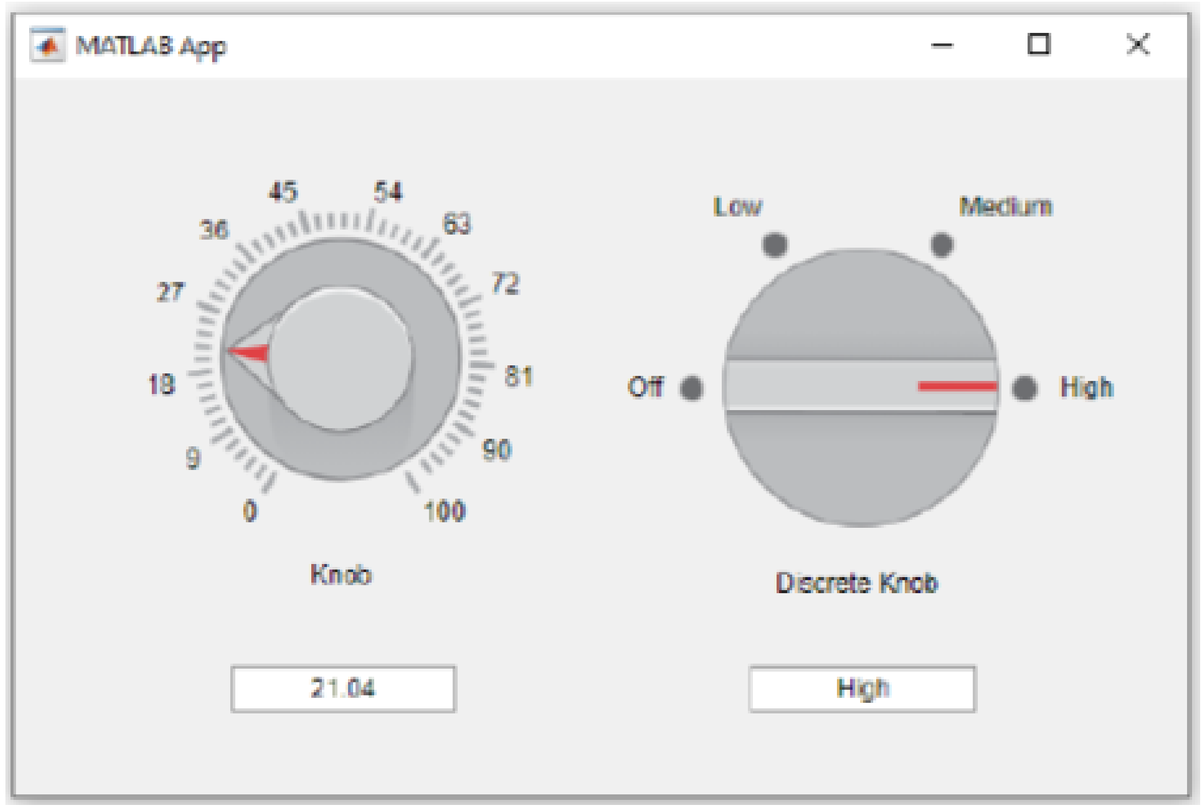

```
function SwitchValueChanged(app, event)
    value = app.Switch.Value;
    if strcmp(value,'On')
        app.Lamp.Color = 'green';
    else
        app.Lamp.Color = 'White';
    end
end

% Value changed function: RockerSwitch
function RockerSwitchValueChanged(app, event)
    value = app.RockerSwitch.Value;
    if strcmp(value,'On')
        app.Lamp.Color = 'red';
    else
        app.Lamp.Color = 'White';
    end
end

% Value changed function: ToggleSwitch
function ToggleSwitchValueChanged(app, event)
    value = app.ToggleSwitch.Value;
    if strcmp(value,'On')
        app.Lamp.Color = 'blue';
    else
        app.Lamp.Color = 'White';
    end
end
```

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Lamp Indicator/Switch/Knob /Gauge




```
        % Value changed function: Knob
        function KnobValueChanged(app, event)
            value = app.Knob.Value;

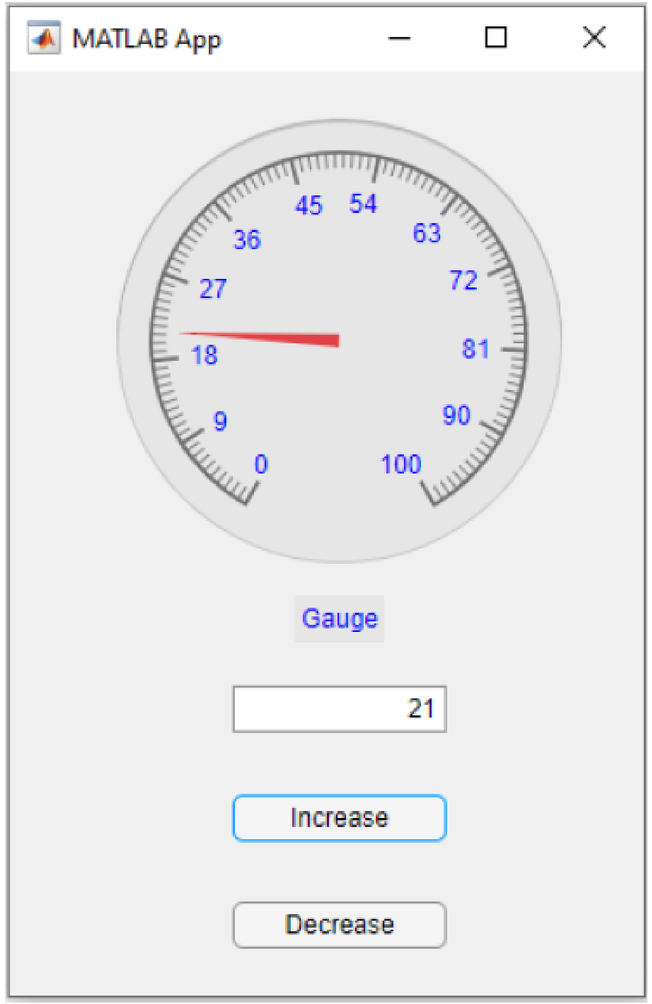
            app.EditField.Value=value;
        end

        % Value changed function: DiscreteKnob
        function DiscreteKnobValueChanged(app, event)
            value = app.DiscreteKnob.Value;

            if strcmp(value,'Off')
                app.EditField_2.Value ='Off';
            elseif strcmp(value,'Low')
                app.EditField_2.Value ='Low';
            elseif strcmp(value,'Medium')
                app.EditField_2.Value ='Medium';
            else
                app.EditField_2.Value ='High';
            end
        end
    end
```

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Lamp Indicator/Switch/Knob /Gauge




```
% Callbacks that handle component events
methods (Access = private)

    % Value changed function: EditField
    function EditFieldValueChanged(app, event)
        value = app.EditField.Value;
        app.Gauge.Value=value;
    end

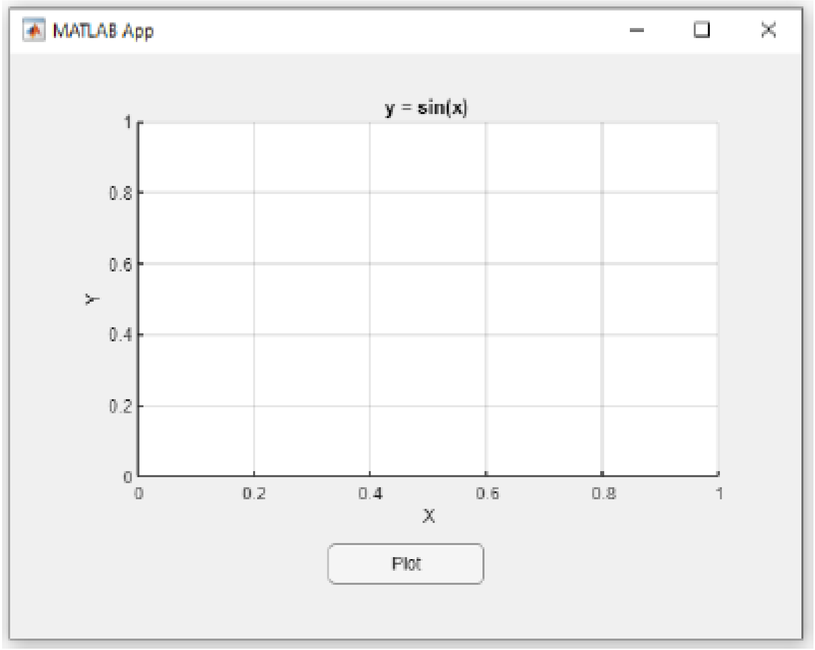
    % Button pushed function: IncreaseButton
    function IncreaseButtonPushed(app, event)
        app.Gauge.Value = app.Gauge.Value +1;
        app.EditField.Value=app.Gauge.Value;
    end

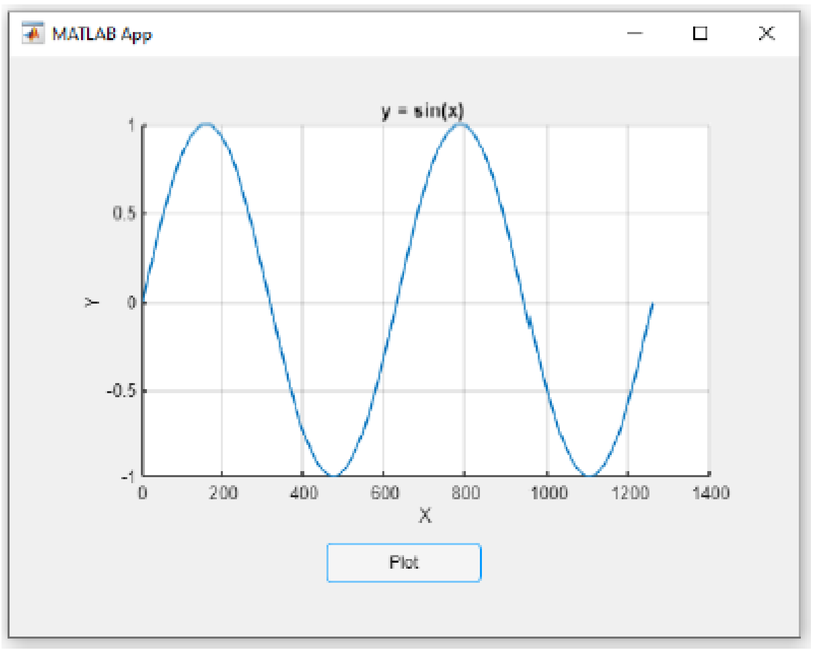
    % Button pushed function: DecreaseButton
    function DecreaseButtonPushed(app, event)
        app.Gauge.Value = app.Gauge.Value -1;
        app.EditField.Value=app.Gauge.Value;
    end
end

% Component initialization
```

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Graph

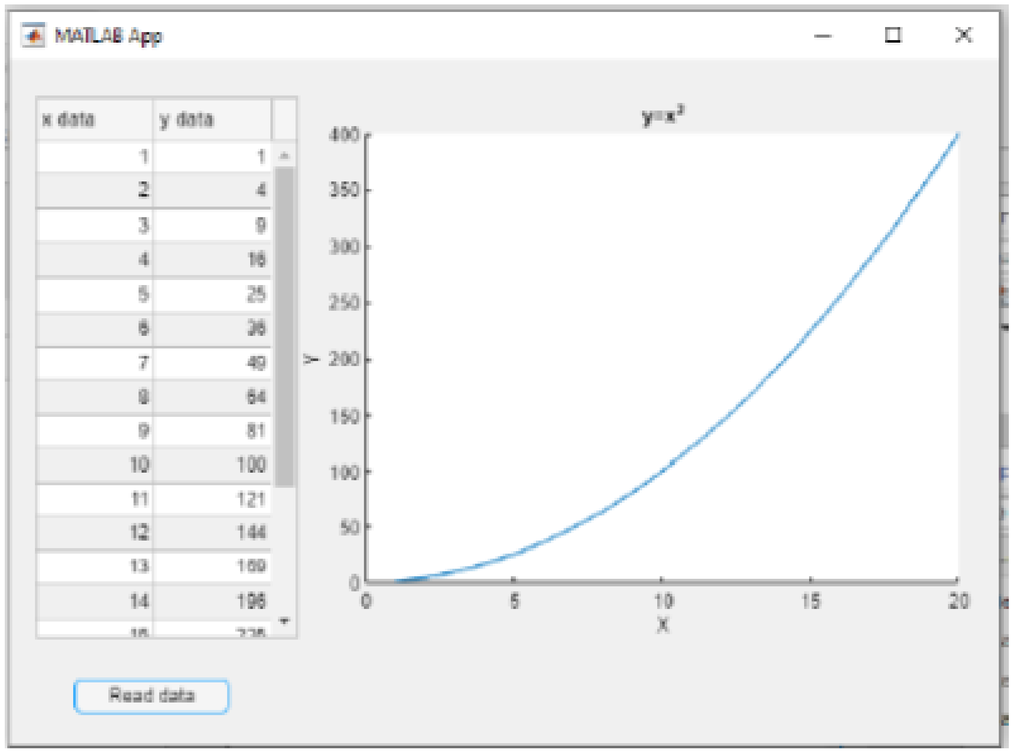





```
x=0:0.01:4*pi;
y=sin(x);
plot(app.UIAxes,y);
```

3.1 พื้นฐานการใช้งานเครื่องมือ AppDesigner

- Import from excel/Graph



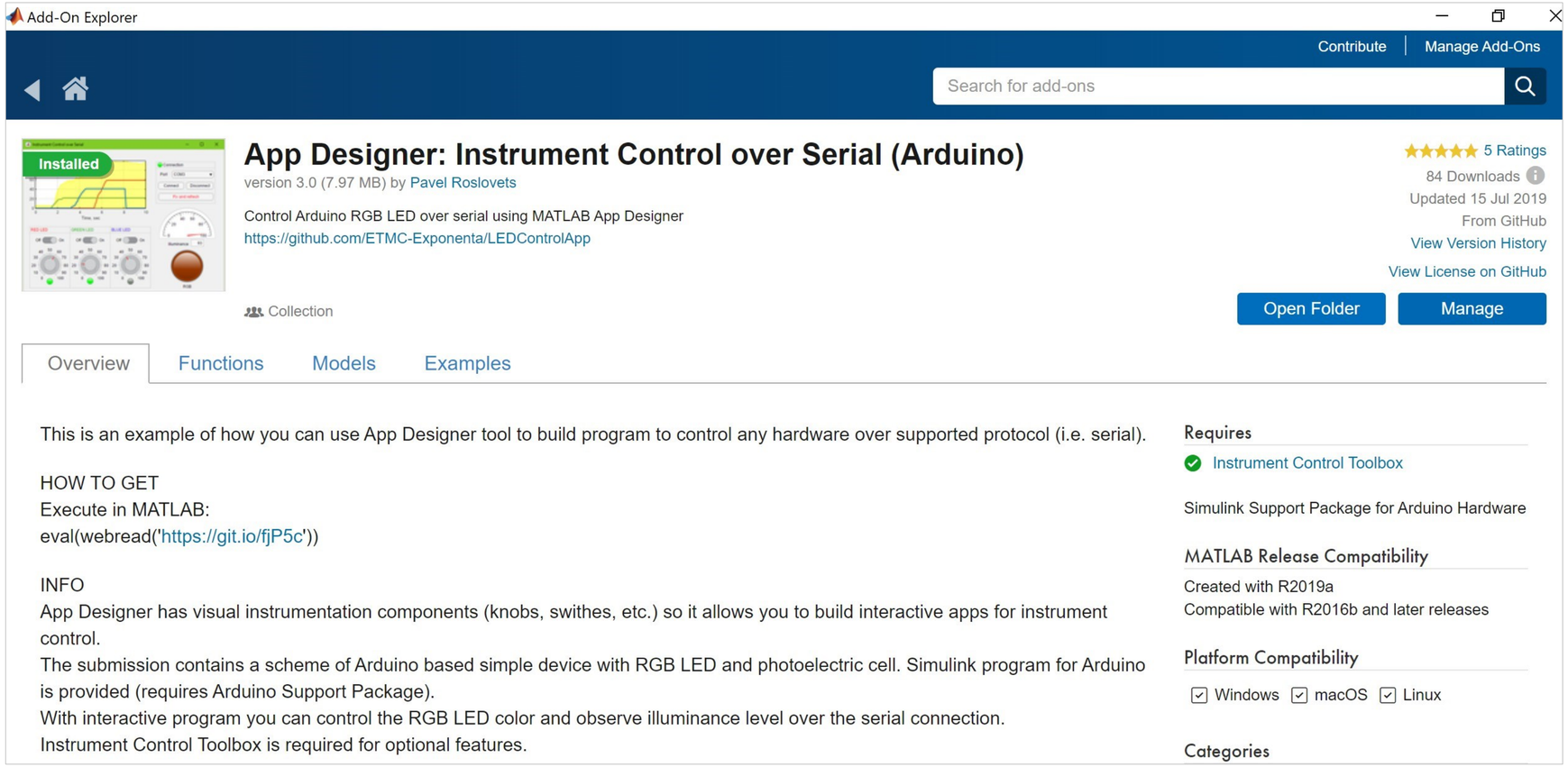
	x	y
2	1	1
3	2	4
4	3	9
5	4	16
6	5	25
7	6	36
8	7	49
9	8	64
10	9	81
11	10	100
12	11	121
13	12	144
14	13	169
15	14	196
16	15	225
17	16	256
18	1	
19	1	
20	1	
21	2	

```
f = readtable("test.xlsx","sheet",1);
app.UITable.Data= f;


f.Properties.VariableNames{1}='x data';
f.Properties.VariableNames{2}='y data';
app.UITable.ColumnName=f.Properties.VariableNames;
x=table2array(f(:,"x data"));
y=table2array(f(:,"y data"));
plot(app.UIAxes,x,y);
```

3.2 ตัวอย่าง Arduino LED Control over Serial

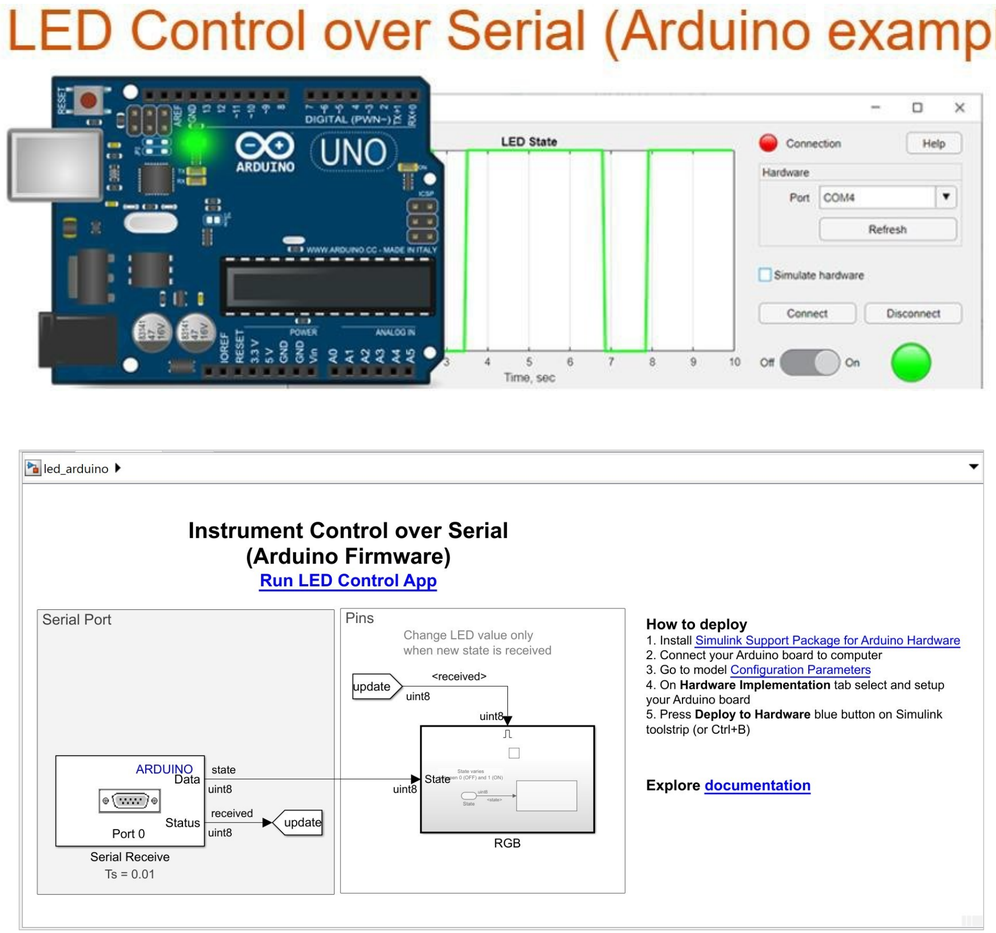
- AppDesinger: Instrument Control over Serial (Arduino)



Ref: https://github.com/ETMC-Exponenta/LEDControlApp

3.2 ตัวอย่าง Arduino LED Control over Serial

- LED Control over Serial (Arduino example)

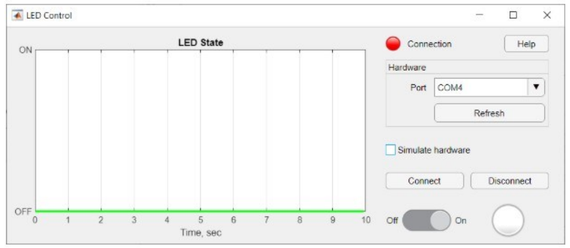


Tutorial

Step 1. Play with LED Control App

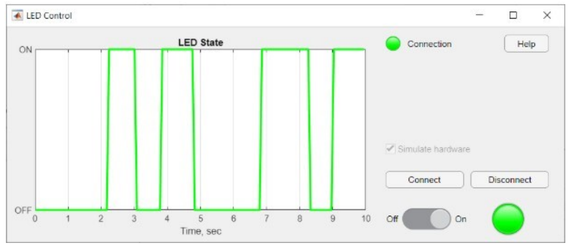
Run LED Control App

```
led_control
```



Check **Simulate Hardware** and press **Connect** button.

After it turn LED switch between **On**/**Off** and observe app behaviour.

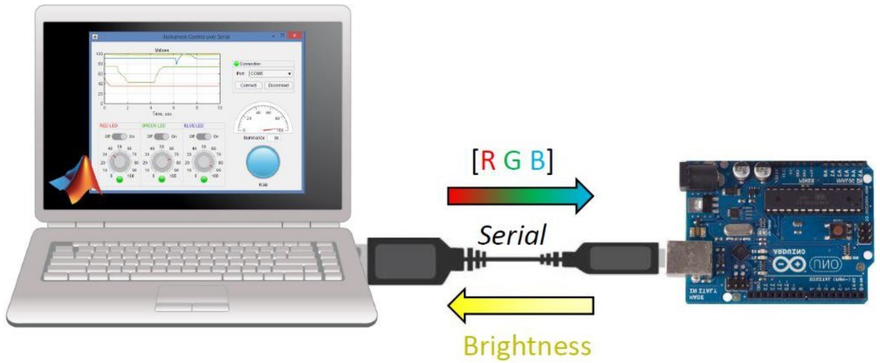


Ref: https://github.com/ETMC-Exponenta/LEDControlApp

3.2 ตัวอย่าง Arduino LED Control over Serial

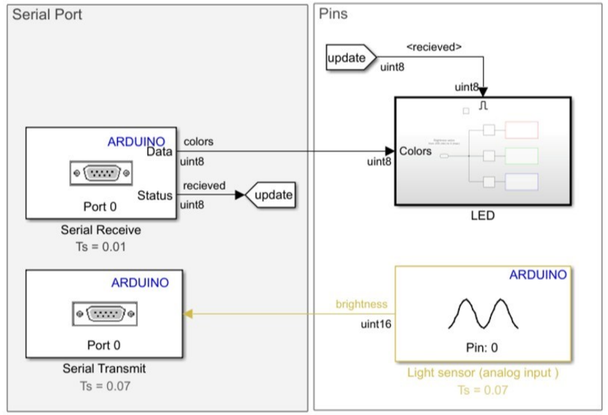
SCHOOL OF ENGINEERING KMITL

- Example RGB Control



Arduino Firmware

Instrument Control over Serial
(Arduino Firmware)
Run Control App



How to deploy
1. Install Simulink Support Package for Arduino Hardware
2. Connect your Arduino board to computer
3. Go to model Configuration Parameters
4. On **Hardware Implementation** tab select and setup your Arduino board
5. Press **Deploy to Hardware** blue button on Simulink toolstrip (or Ctrl+B)

Open Arduino firmware model:

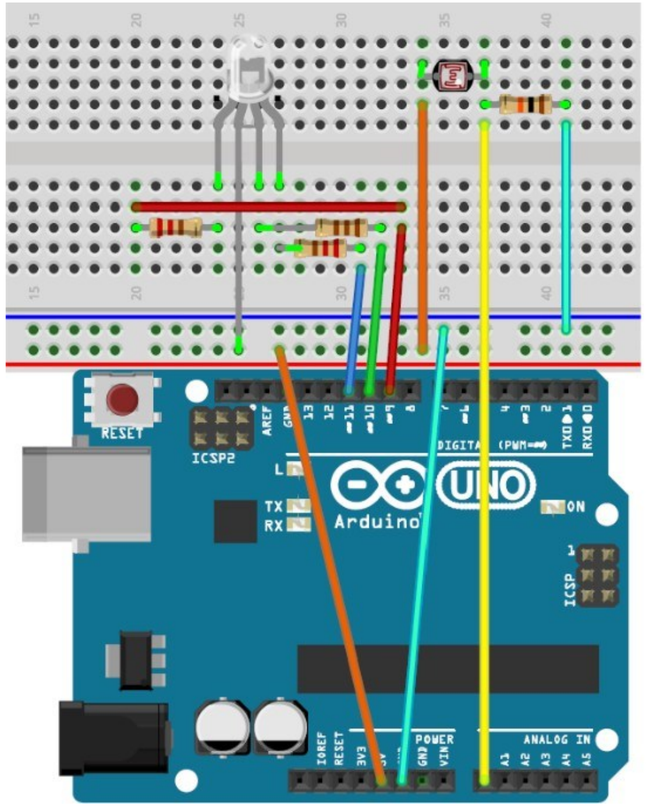
```
rgb_arduino
```

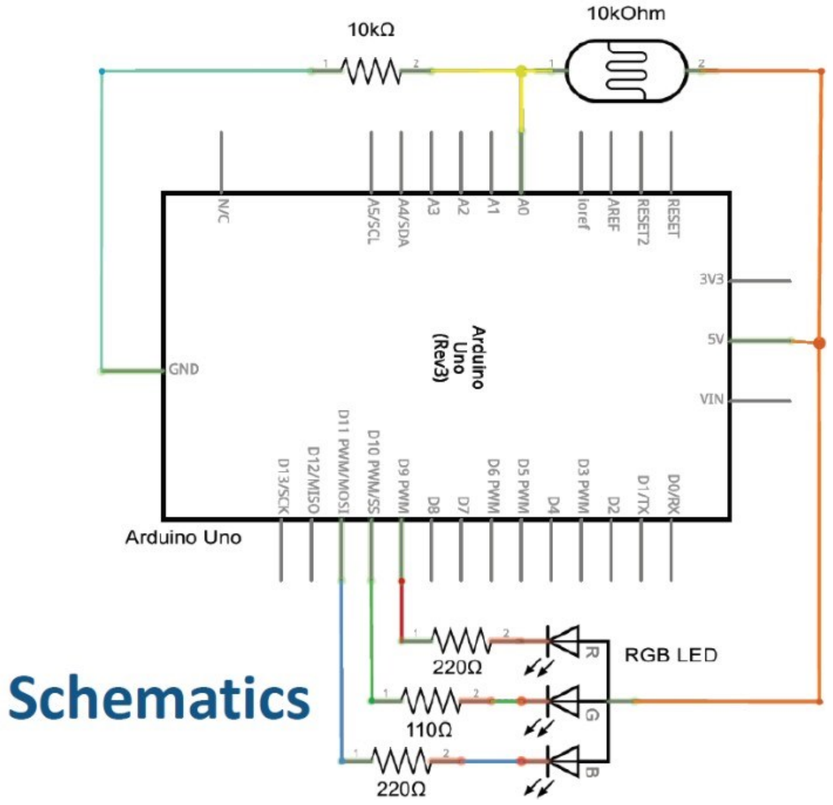
Follow **How to deploy** steps to deploy model to Arduino.

Ref: https://github.com/ETMC-Exponenta/LEDControlApp

3.2 ตัวอย่าง Arduino LED Control over Serial

- Example RGB Control





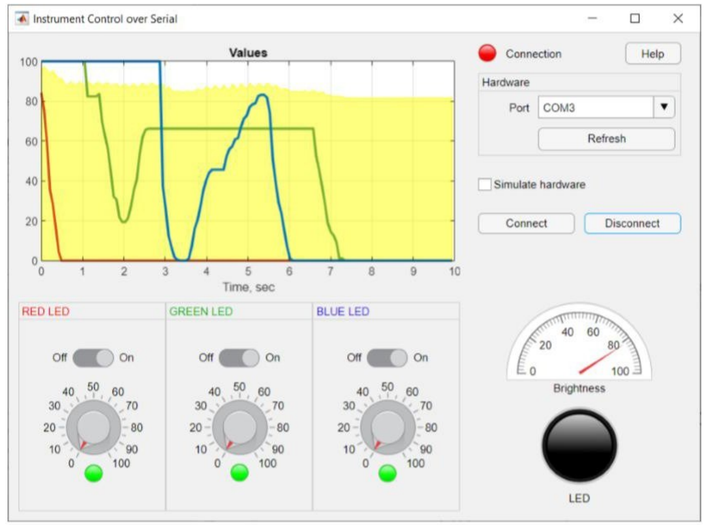
Schematics

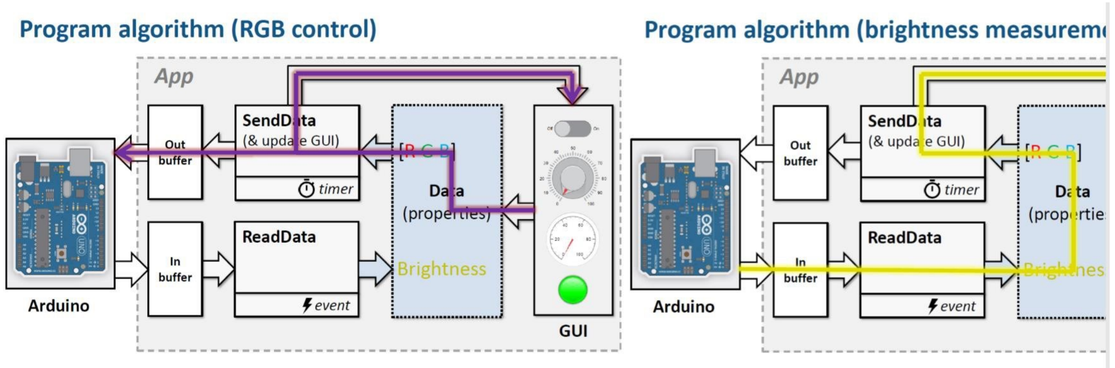
Ref: https://github.com/ETMC-Exponenta/LEDControlApp

3.2 ตัวอย่าง Arduino LED Control over Serial

- Example RGB Control

RGB Control App





Run Control App:

```
rgb_control
```

How to use:

1. Select or type in Arduino COM **Port** (see how to find Arduino port)
2. OR check **Simulate hardware** if you don't want to use serial and only want to explore App behaviour
3. Press **Connect** to open connection and start control process
4. Press **Disconnect** or close the App to stop control process

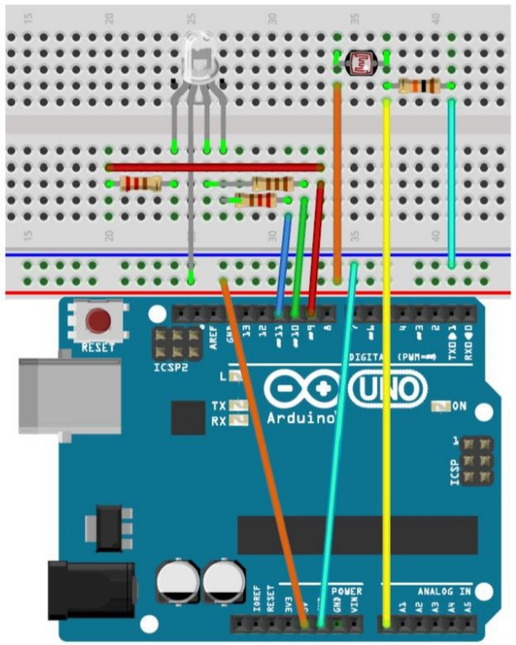
Ref: https://www.mathworks.com/matlabcentral/fileexchange/57168-app-designer-instrument-control-over-serial-arduino

3.2 ตัวอย่าง Arduino LED Control over Serial

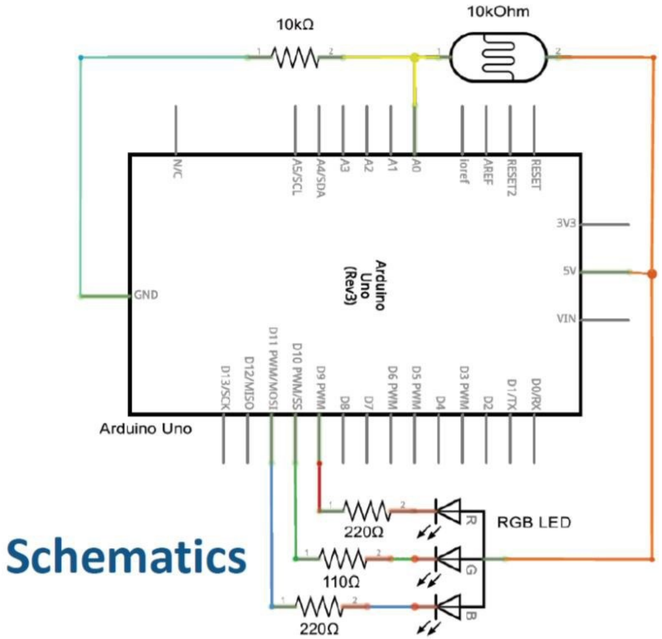
- Example RGB Control

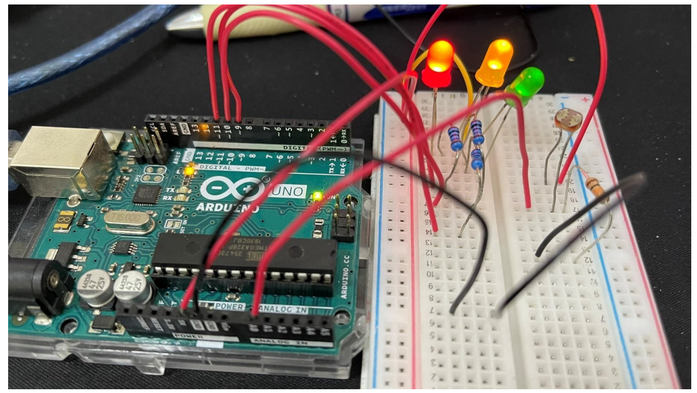
Arduino Hardware

Connect RGB LED to Arduino as shown on the scheme



Example with DIY shield

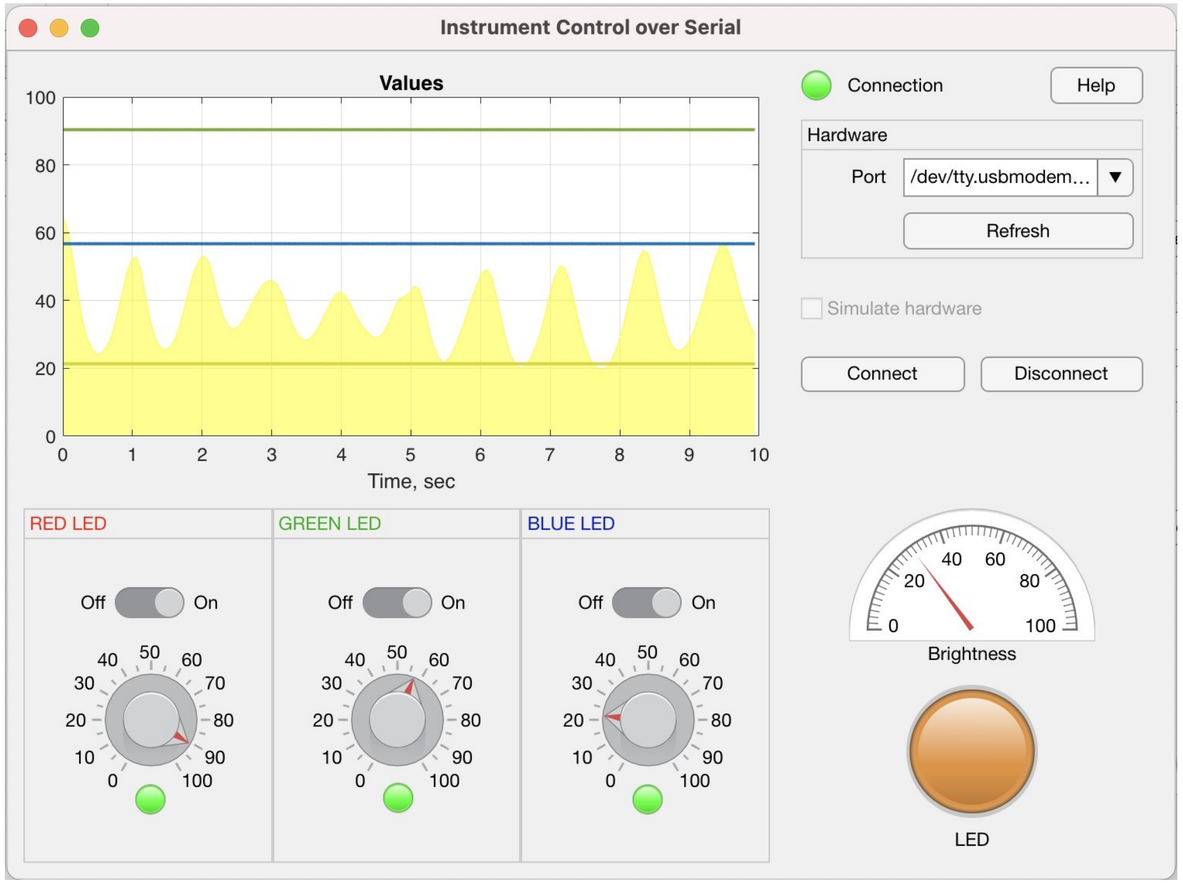


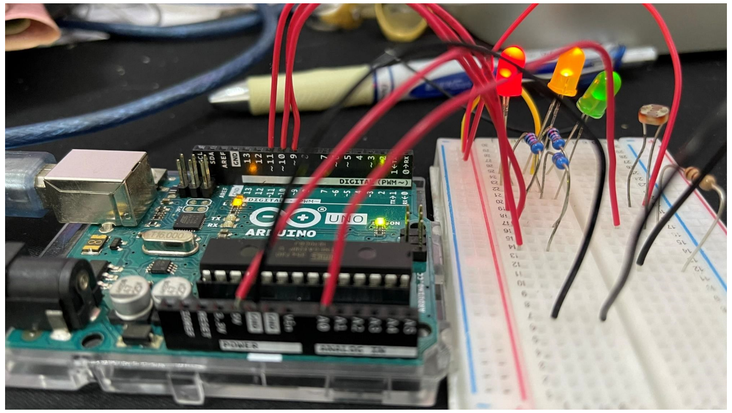


Ref: https://www.mathworks.com/matlabcentral/fileexchange/57168-app-designer-instrument-control-over-serial-arduino

3.2 ตัวอย่าง Arduino LED Control over Serial

- Example RGB Control

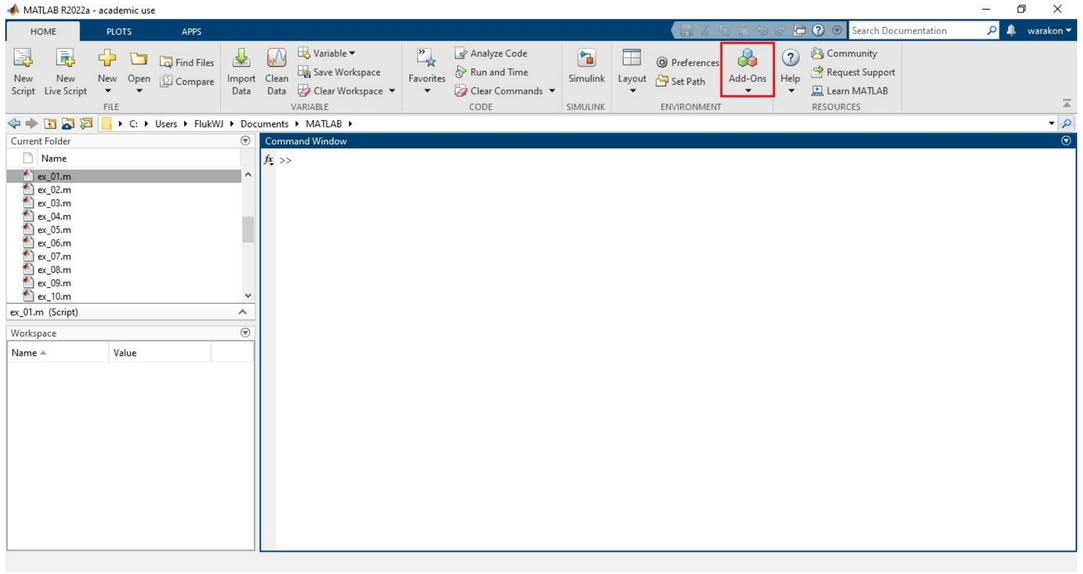


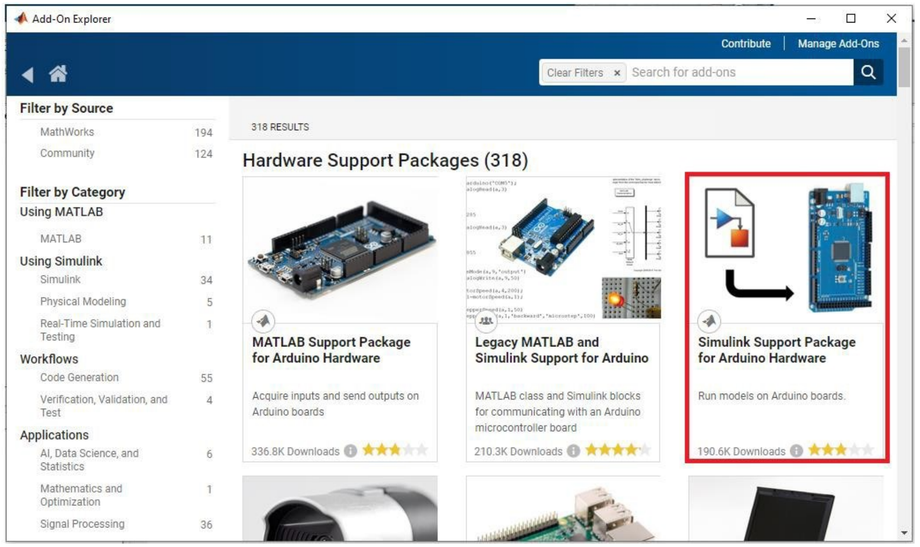


Ref: https://www.mathworks.com/matlabcentral/fileexchange/57168-app-designer-instrument-control-over-serial-arduino

3.3 การติดตั้ง Simulink Support Package for Arduino Hardware

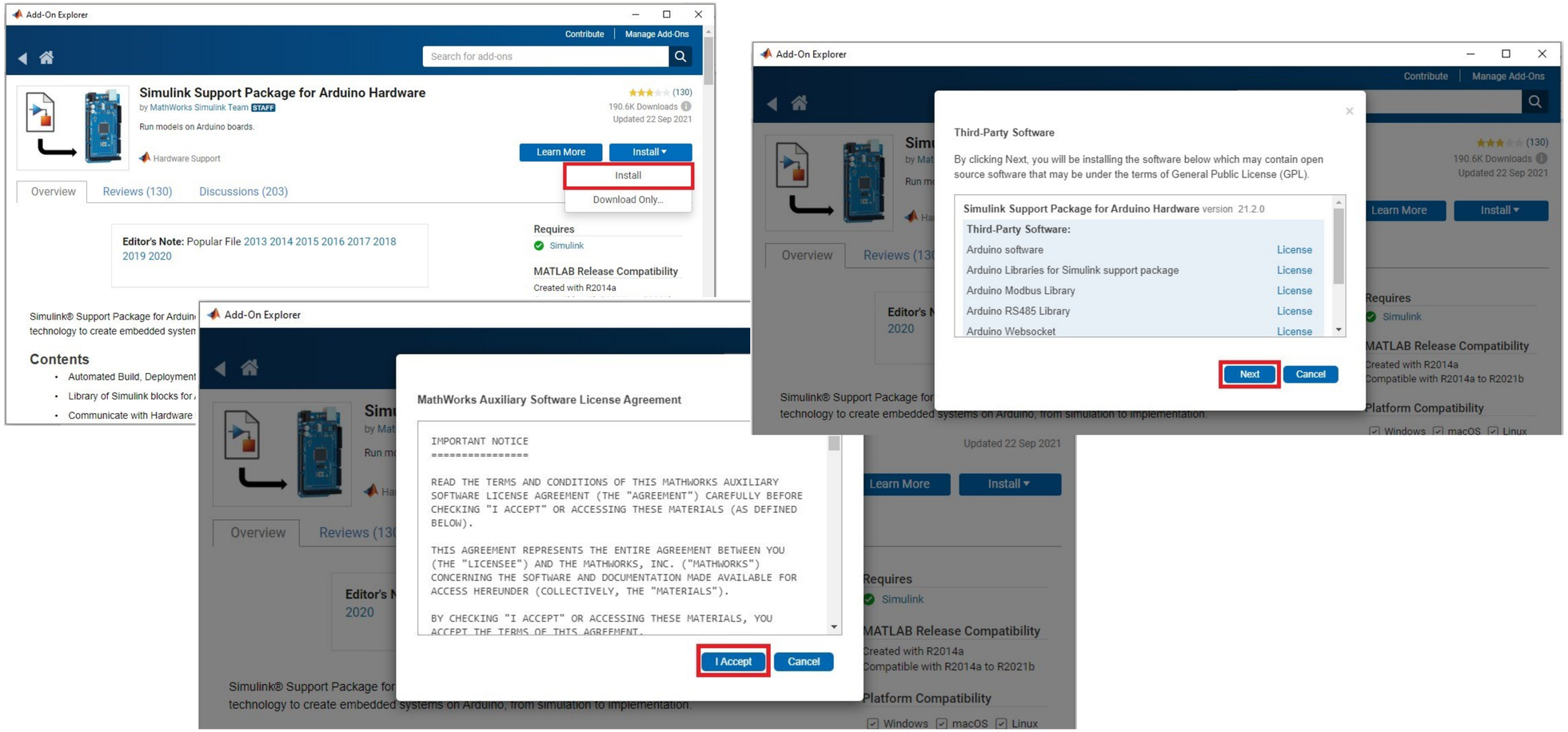
- Simulink® Support Package for Arduino® hardware will empower you with the latest Model-Based Design technology to create embedded systems on Arduino, from simulation to implementation.





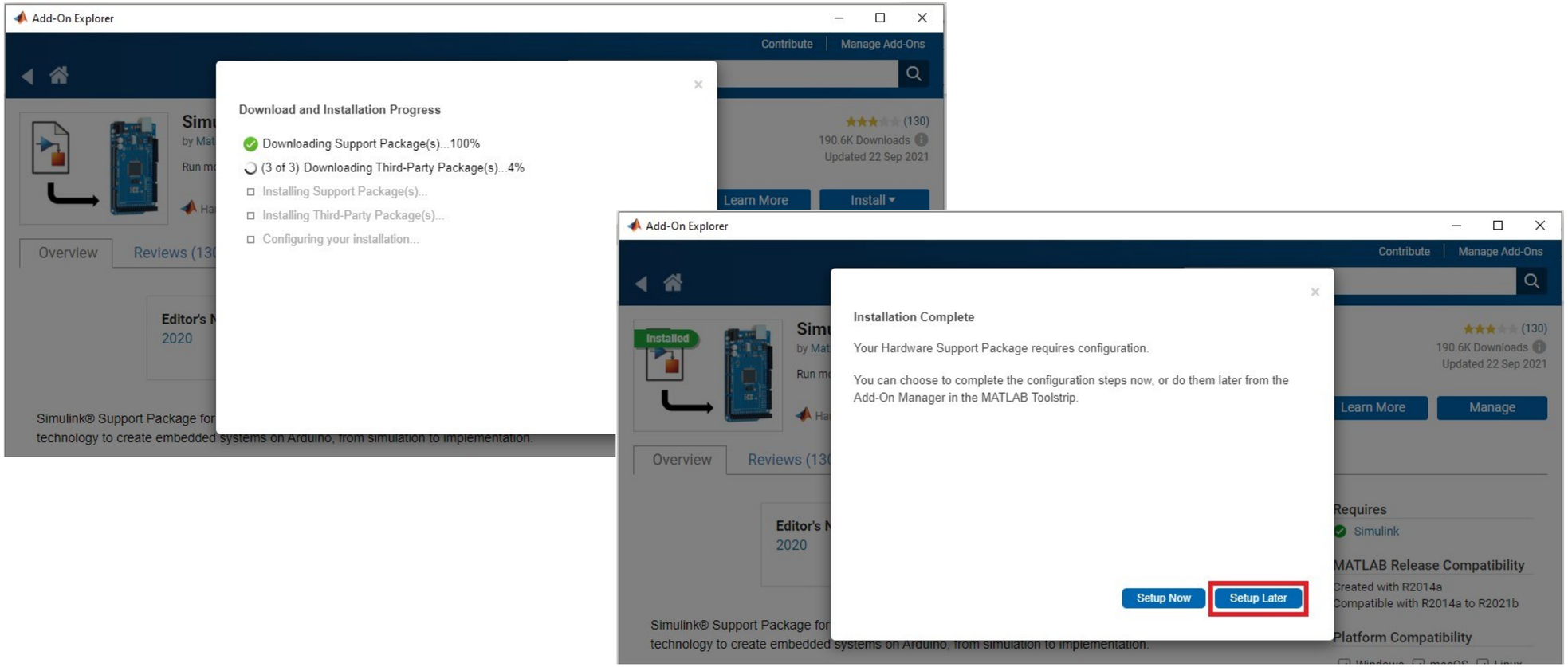
3.3 การติดตั้ง Simulink Support Package for Arduino Hardware

- Simulink® Support Package for Arduino® hardware



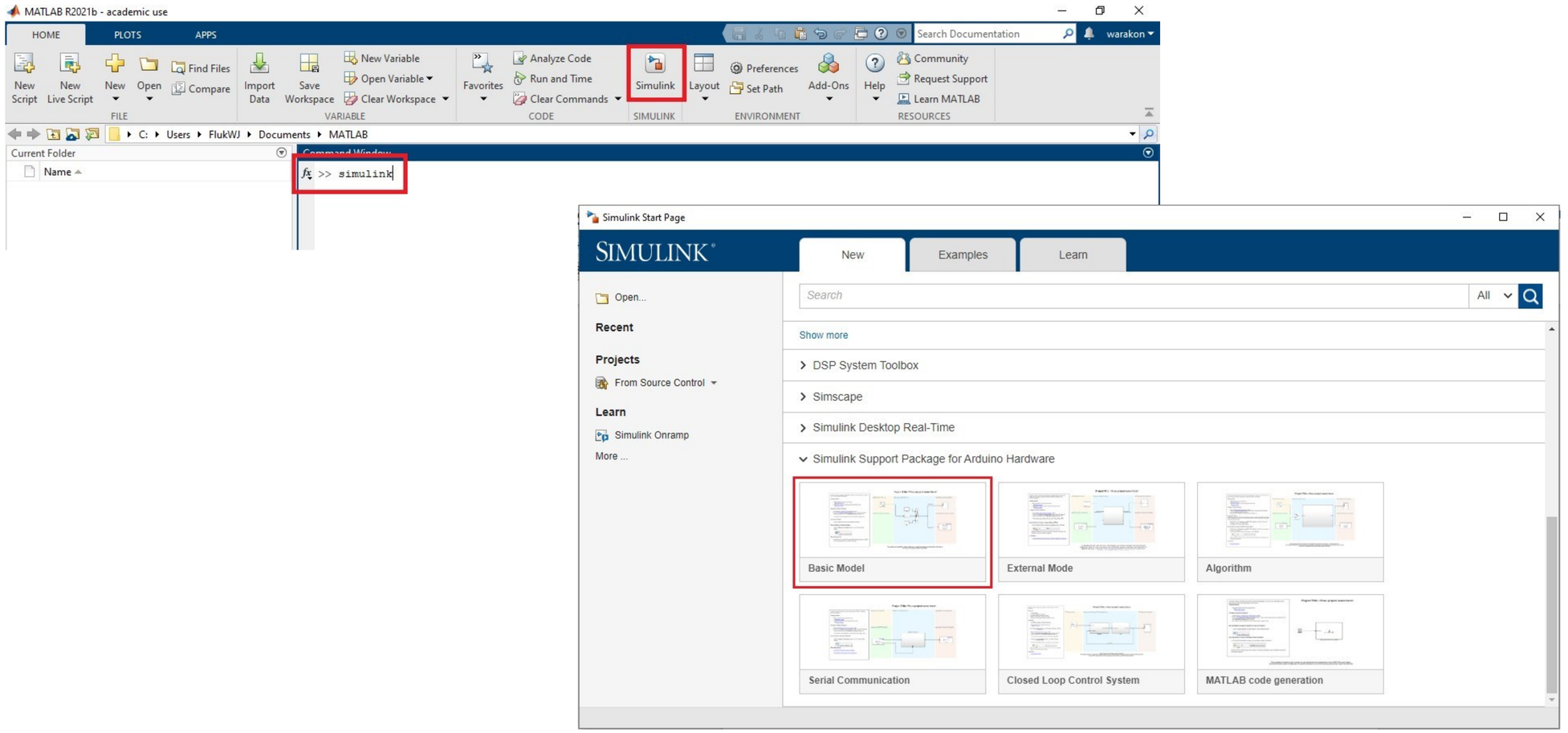
3.3 การติดตั้ง Simulink Support Package for Arduino Hardware

- Simulink® Support Package for Arduino® hardware



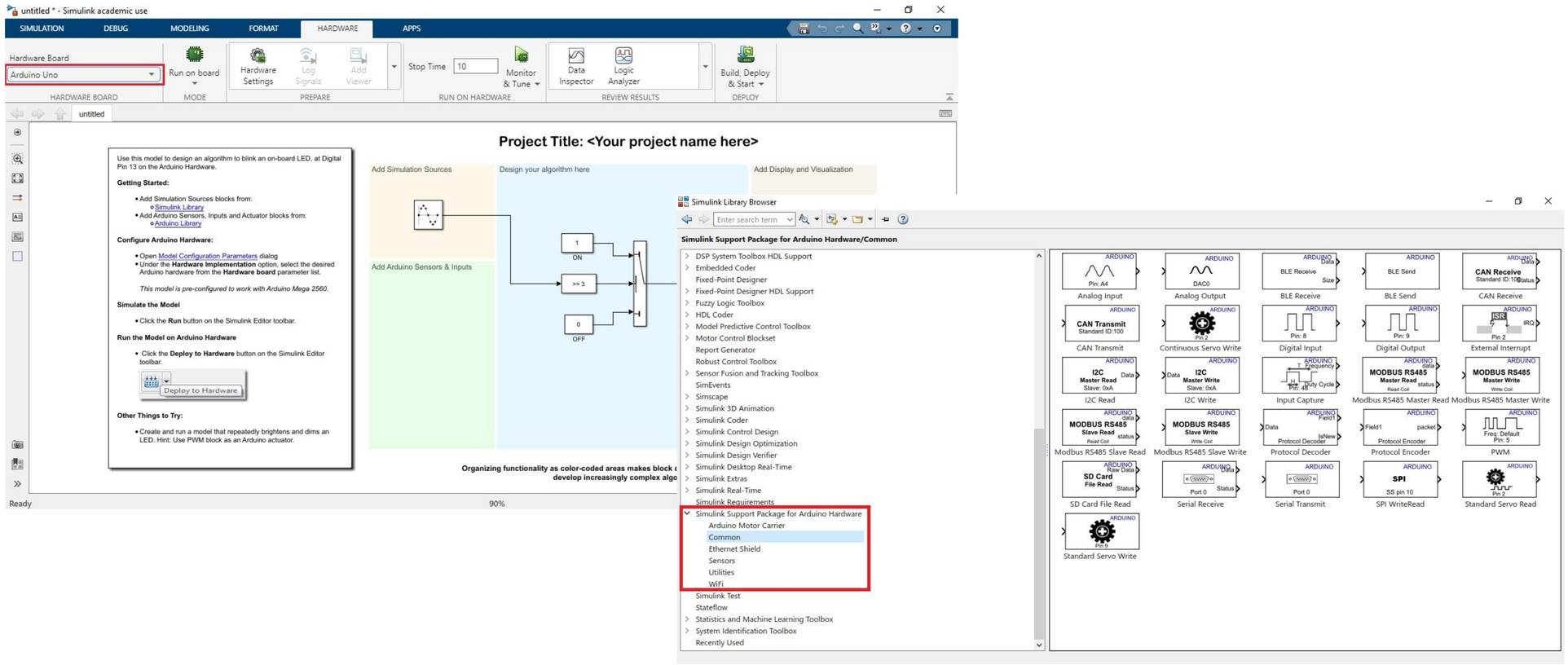
3.3 การติดตั้ง Simulink Support Package for Arduino Hardware

- Simulink® Support Package for Arduino® hardware



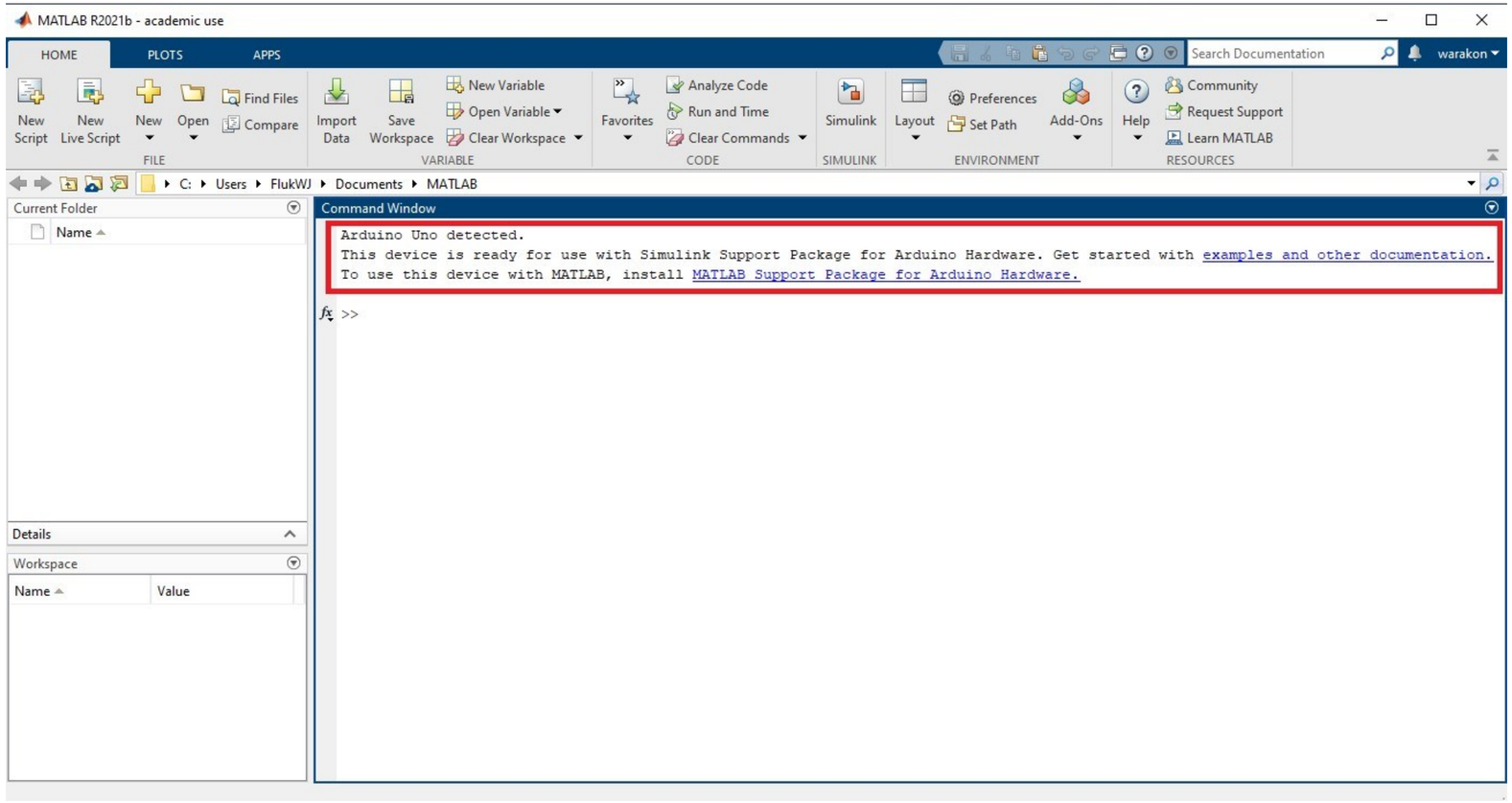
3.3 การติดตั้ง Simulink Support Package for Arduino Hardware

- Simulink® Support Package for Arduino® hardware



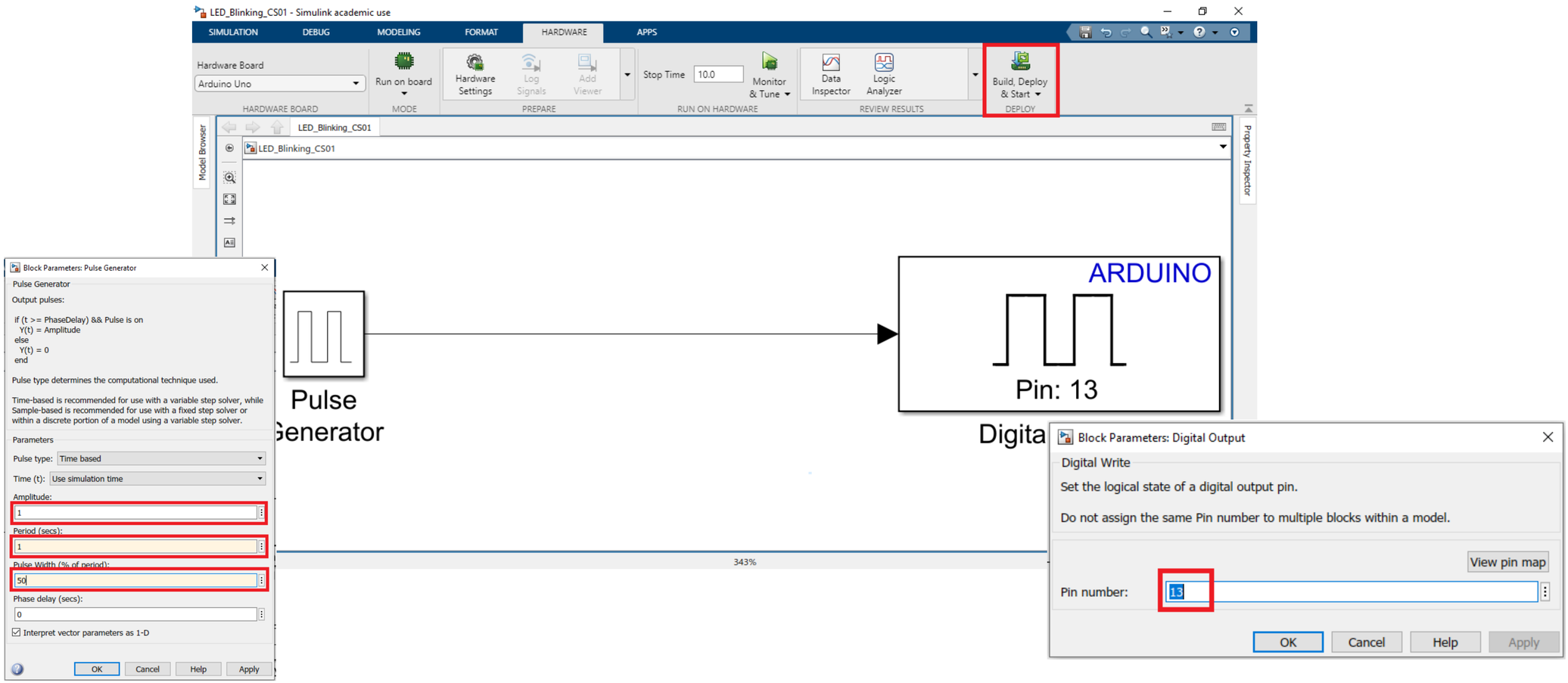
3.3 การติดตั้ง Simulink Support Package for Arduino Hardware

- Simulink® Support Package for Arduino® hardware



3.3 การติดตั้ง Simulink Support Package for Arduino Hardware

- Simulink® Support Package for Arduino® hardware

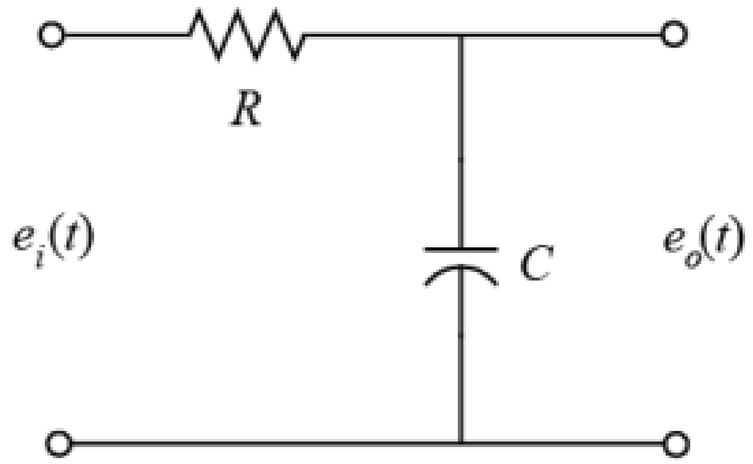


3.4 การทดลองควบคุมระบบไฟฟ้า (Electrical system)

SCHOOL OF ENGINEERING KMITL

- Electrical system

ระบบวงจรไฟฟ้า RC



โมเดลทางคณิตศาสตร์ (First-order)

KVL; $e_i(t) - Ri(t) - \frac{1}{C}\int_0^t i(t)dt = 0$

Laplace transform

$$E_i(s) - RI(s) - \frac{1}{Cs}I(s) = 0$$

$$E_i(s) = \left(R + \frac{1}{Cs}\right)I(s)$$

Output: $e_o(t) = \frac{1}{C}\int_0^t i(t)dt$

Laplace transform

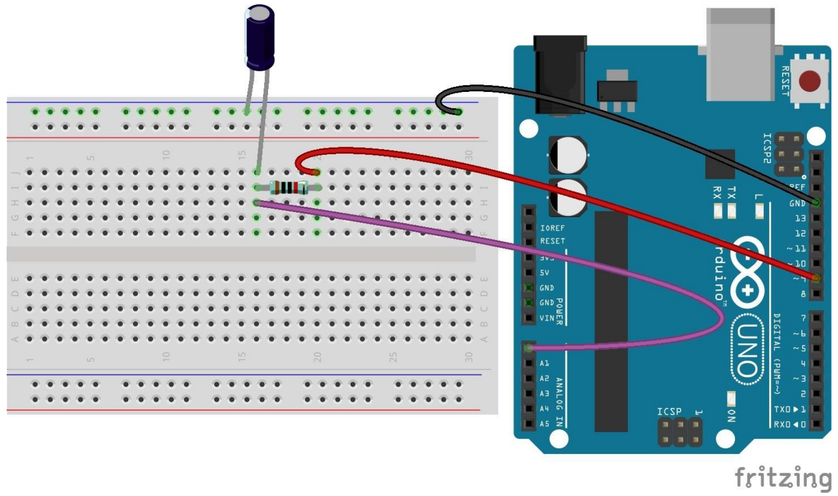
$$E_o(s) = \frac{1}{Cs}I(s)$$

Transfer function $\frac{E_o(s)}{E_i(s)}$

$$\frac{E_o(s)}{E_i(s)} = \frac{1}{RCs+1}$$

3.4 การทดลองควบคุมระบบไฟฟ้า (Electrical system)

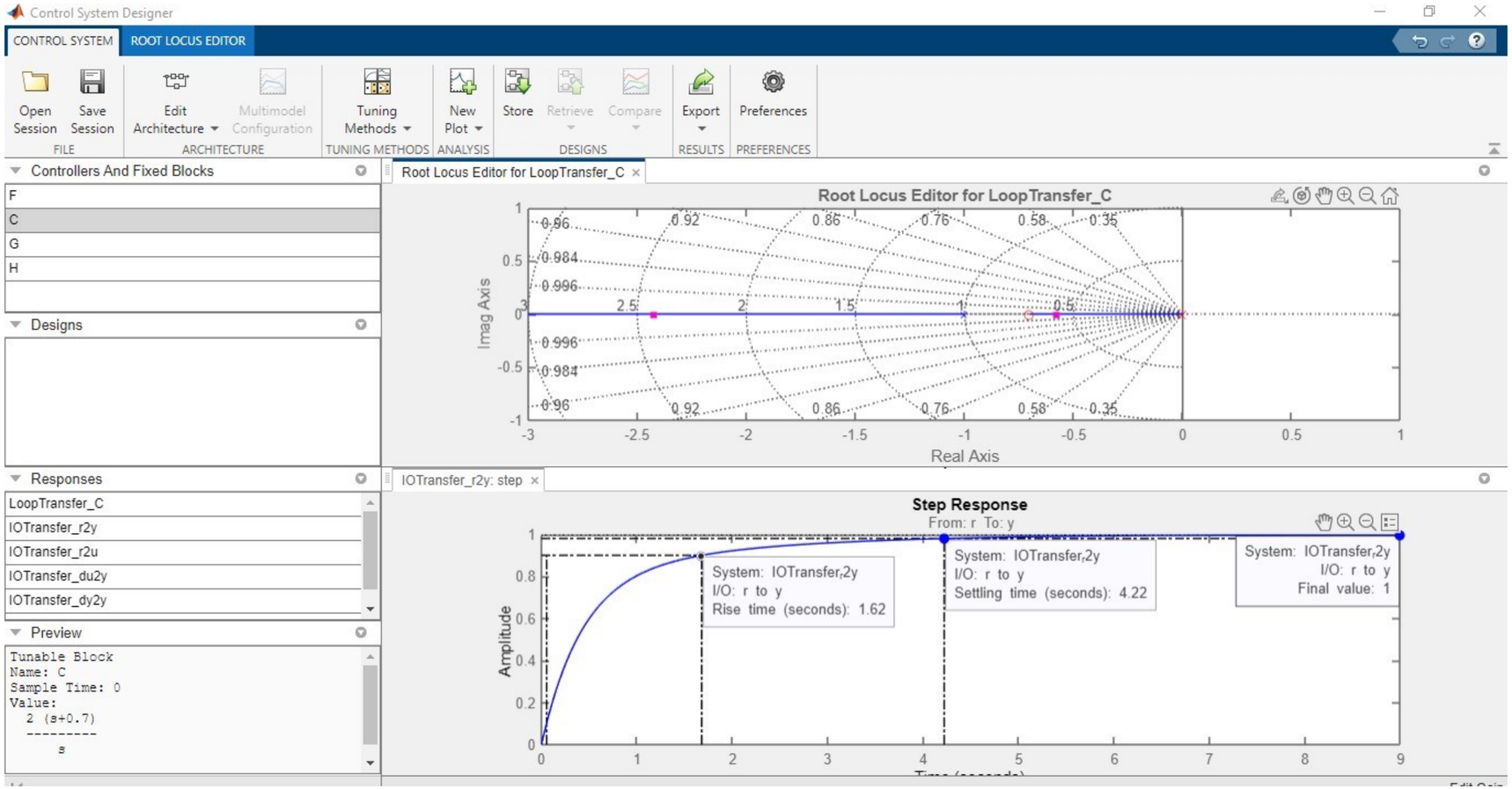
- Wiring



เครื่องมือและอุปกรณ์	จำนวน
1. ตัวต้านทาน $10k\Omega$	1
2. ตัวเก็บประจุ $100\mu F$	1
3. โพโต้บอร์ด	1
4. บอร์ด Arduino Uno R3 พร้อม USB	1
5. สาย Jumper	3
6. คอมพิวเตอร์	1

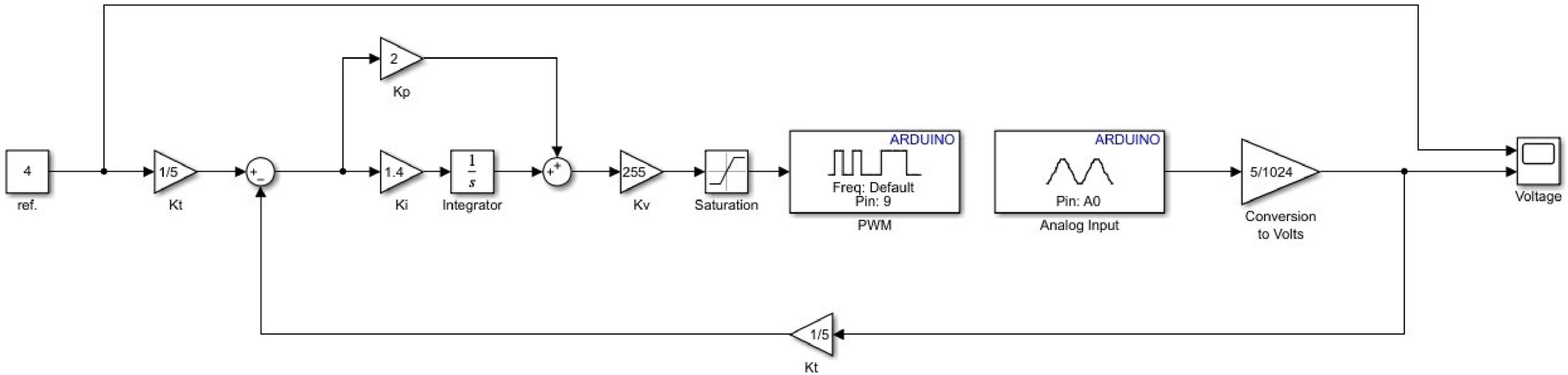
3.4 การทดลองควบคุมระบบไฟฟ้า (Electrical system)

- SISO Tool Design (Root Locus)



3.4 การทดลองควบคุมระบบไฟฟ้า (Electrical system)

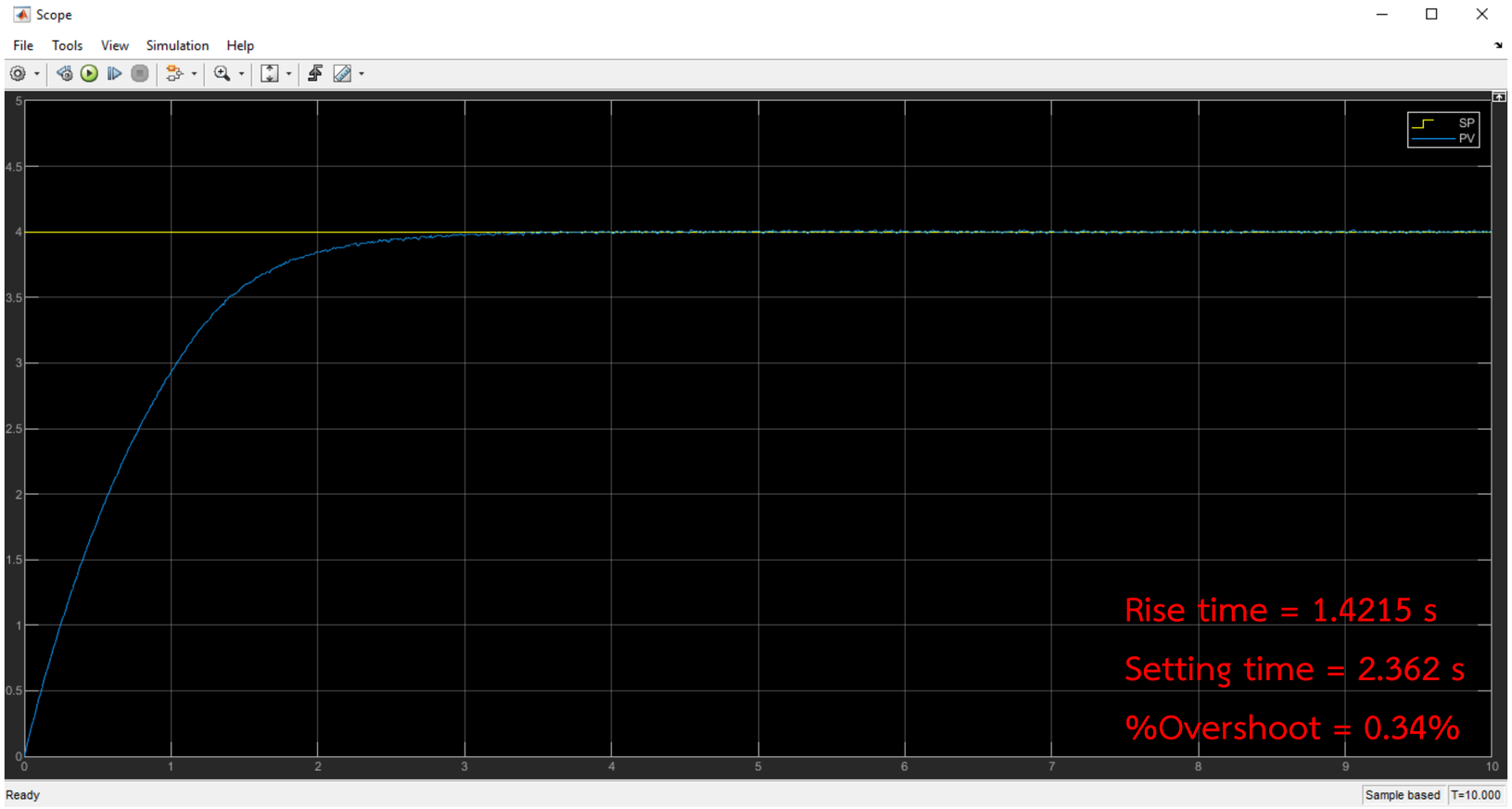
- Hardware-in-the-loop



3.4 การทดลองควบคุมระบบไฟฟ้า (Electrical system)

SCHOOL OF ENGINEERING KMITL

- Response

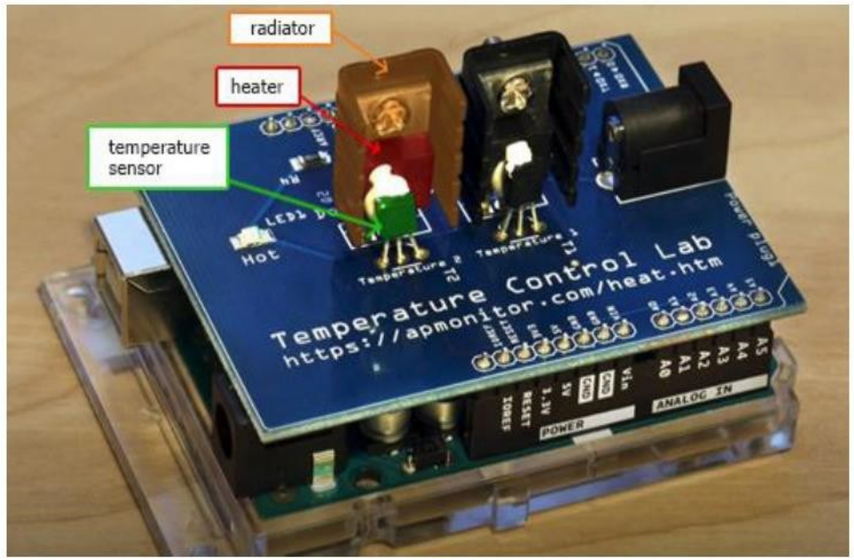


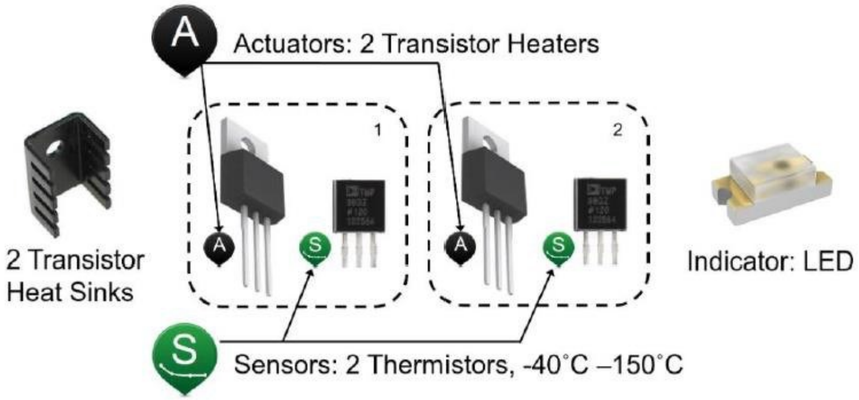
3.5 การทดลองควบคุมอุณภูมิ

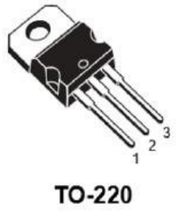
- Temperature control

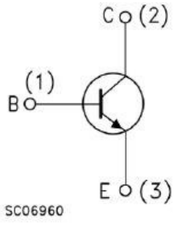


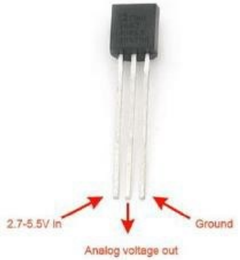






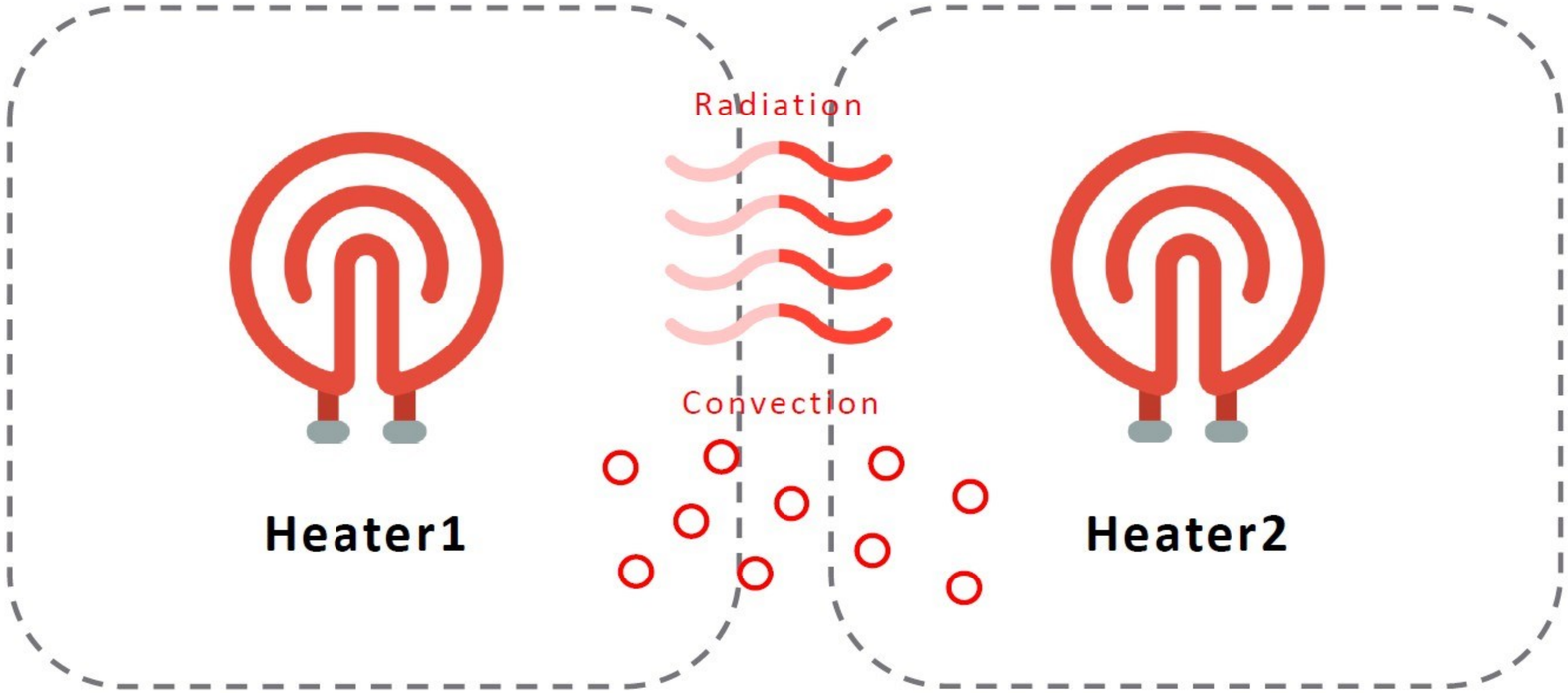






3.5 การทดลองควบคุมอุณภูมิ

- Temperature control



Input Q1(Power 0-100%)

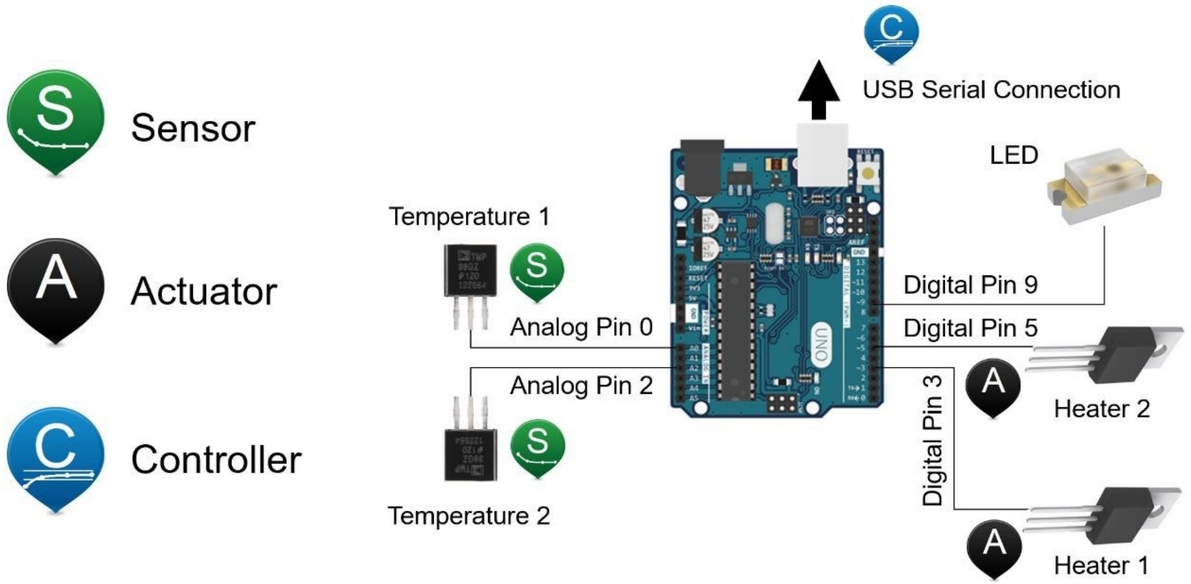
Output T1(Temperature 30-100 °C)

Input Q2(Power 0-100%)

Output T2(Temperature 30-100 °C)

3.5 การทดลองควบคุมอุณภูมิ

- Wiring





3.5 การทดลองควบคุมอุณภูมิ

- Mathematical Model

ระบบควบคุมอุณหภูมิใช้โมเดลแบบไม่เป็นเชิงเส้นลาดับสอง โดยมีกฮีตเตอร์ 1 และฮีตเตอร์ 2 (Q1 และ Q2) พร้อมกับอุณหภูมิ 1 และ 2 (Tc1, Tc2, Th1, Th2) ความแตกต่างระหว่างอุณหภูมิของฮีตเตอร์ทั้งสอง (ΔTh=Th2-Th1) ถูกใช้เพื่อประมาณการการถ่ายเทความร้อนระหว่างฮีตเตอร์ทั้งสอง สมการสมดุลพลังงานสองสมการอธิบายการตอบสนองของอุณหภูมิแบบไดนามิกของเซ็นเซอร์อุณหภูมิทั้งสองเมื่อมีการป้อนค่า อินพุตจากฮีตเตอร์ มีดังนี้

Non-Linear Model

Convection to ambient

Convection interaction

$$mc_p \frac{dT_1}{dt} = UA(T_a - T_1) + \varepsilon\sigma A(T_a^4 - T_1^4) + UA_s(T_2 - T_1) + \varepsilon\sigma A_s(T_2^4 - T_1^4) + \alpha_1 Q_1$$

$$mc_p \frac{dT_2}{dt} = UA(T_a - T_2) + \varepsilon\sigma A(T_a^4 - T_2^4) + UA_s(T_1 - T_2) + \varepsilon\sigma A_s(T_1^4 - T_2^4) + \alpha_2 Q_2$$

Radiation to ambient

Radiation interaction

3.5 การทดลองควบคุมอุณภูมิ

SCHOOL OF ENGINEERING KMITL

- Mathematical Model

Quantity	Value
Initial temperature (T_0)	303.15 K $(30^\circ C)$
Ambient temperature (T_∞)	303.15 K $(30^\circ C)$
Heater output (Q)	0 to 1 W
Heater factor (α_1)	0.005 W/(% heater)
Heater factor (α_2)	0.002 W/(% heater)
Heat capacity (C_p)	500 J/kg-K
Surface Area (A)	$1.2\text{x}10^{-3}$ m^2 $(12cm^2)$
Mass (m)	0.004 kg $(4gm)$
Overall Heat Transfer Coefficient (U)	$4.1365\ W/m^2-K$
Emissivity (ε)	0.9
Stefan Boltzmann Constant (σ)	$5.67\text{x}10^{-8}\ W/m^2-K^4$

3.5 การทดลองควบคุมอุณภูมิ

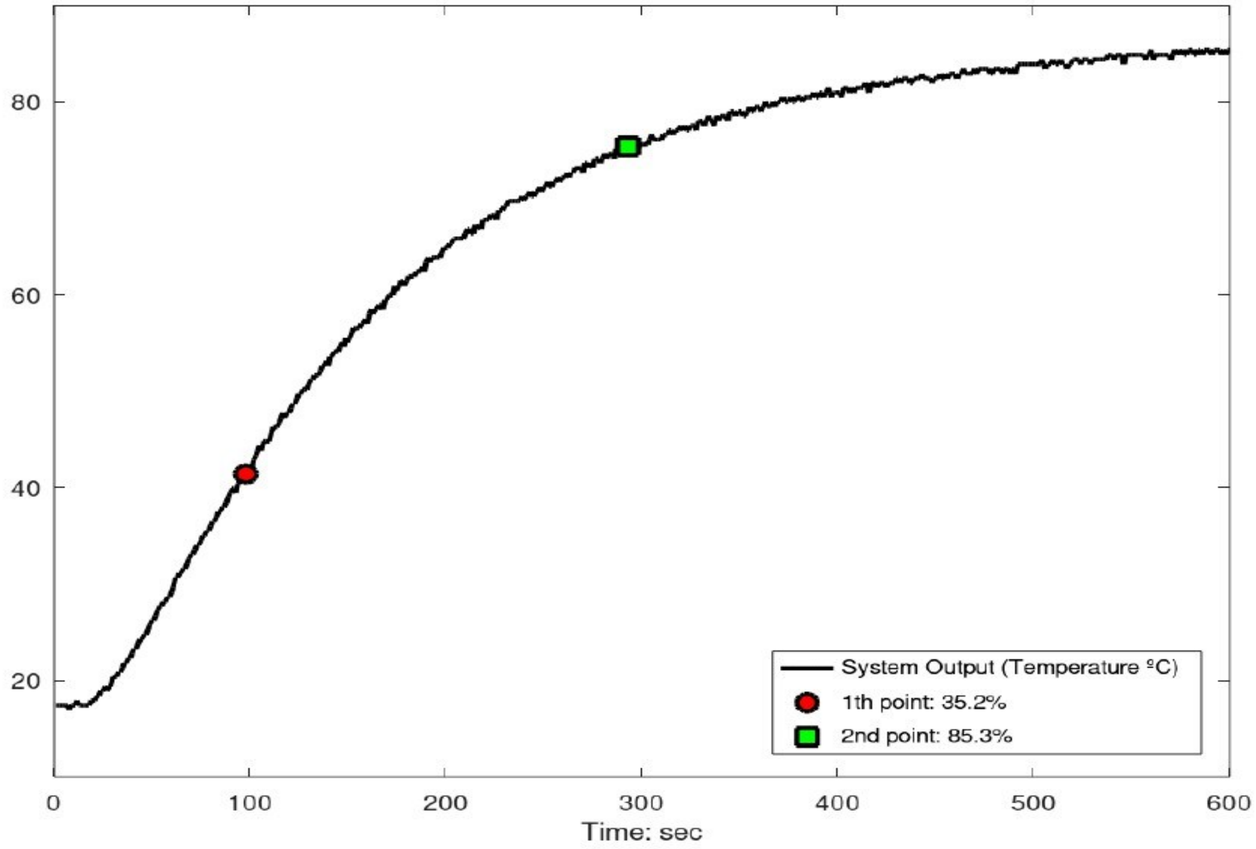
- Mathematical Model : first order plus time delay model

$$G_p(s) = K\frac{1}{(1+sT)}e^{-sL}$$

$$K = \frac{y_{ss} - y_0}{u_{ss} - u_0}$$

$$T = 0.67(t_{85} - t_{35})$$

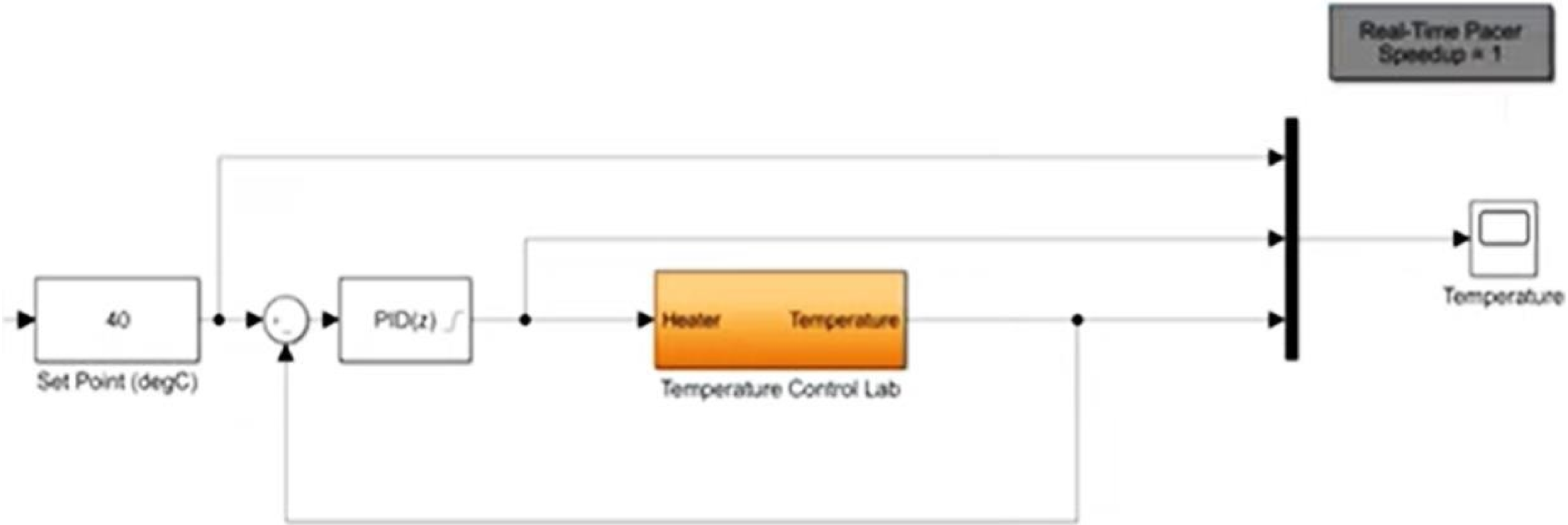
$$L = (1.3 \times t_{85} - 0.29 \times t_{35})$$



Open-loop step response obtained with TCLab. Two-points are outlined corresponding 35.2% and 85.3% of the final steady-state output values.

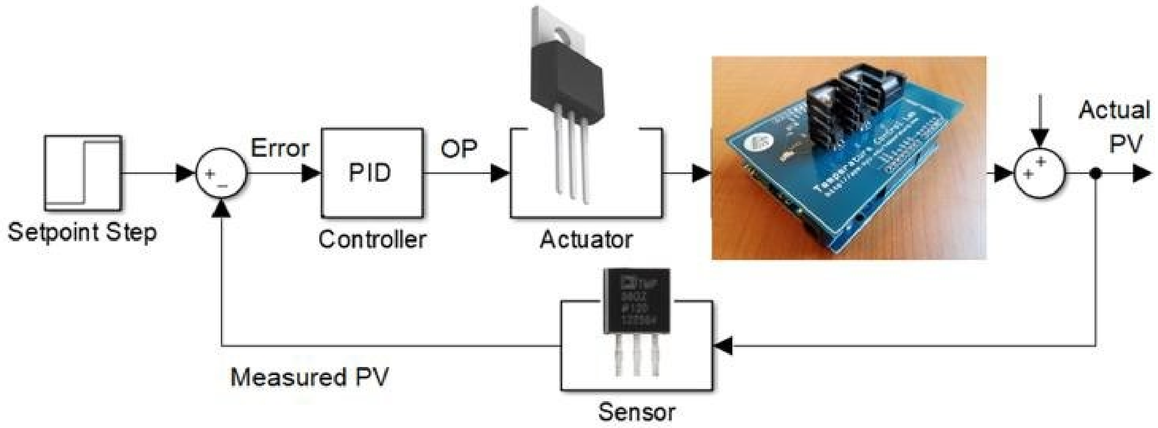
3.5 การทดลองควบคุมอุณภูมิ

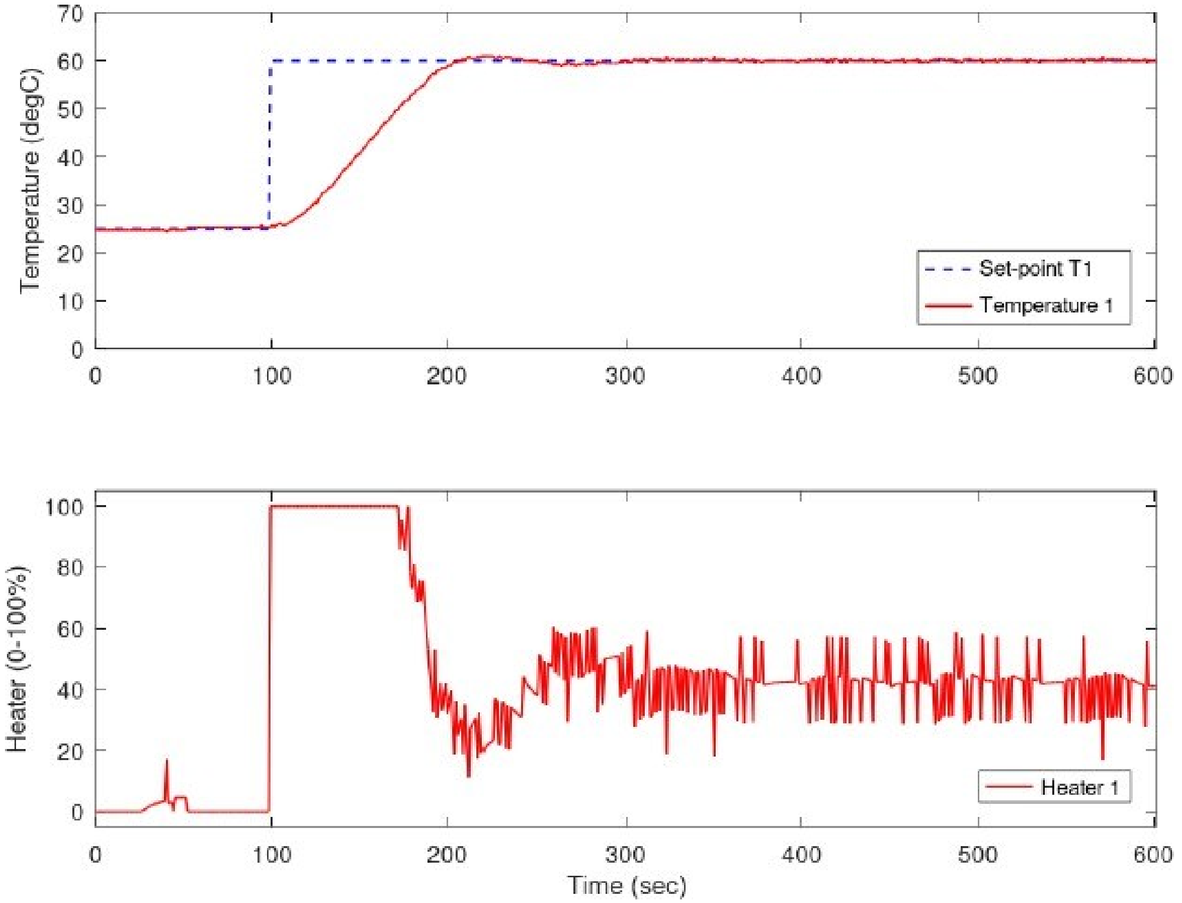
- PID control



3.5 การทดลองควบคุมอุณภูมิ

- PID control



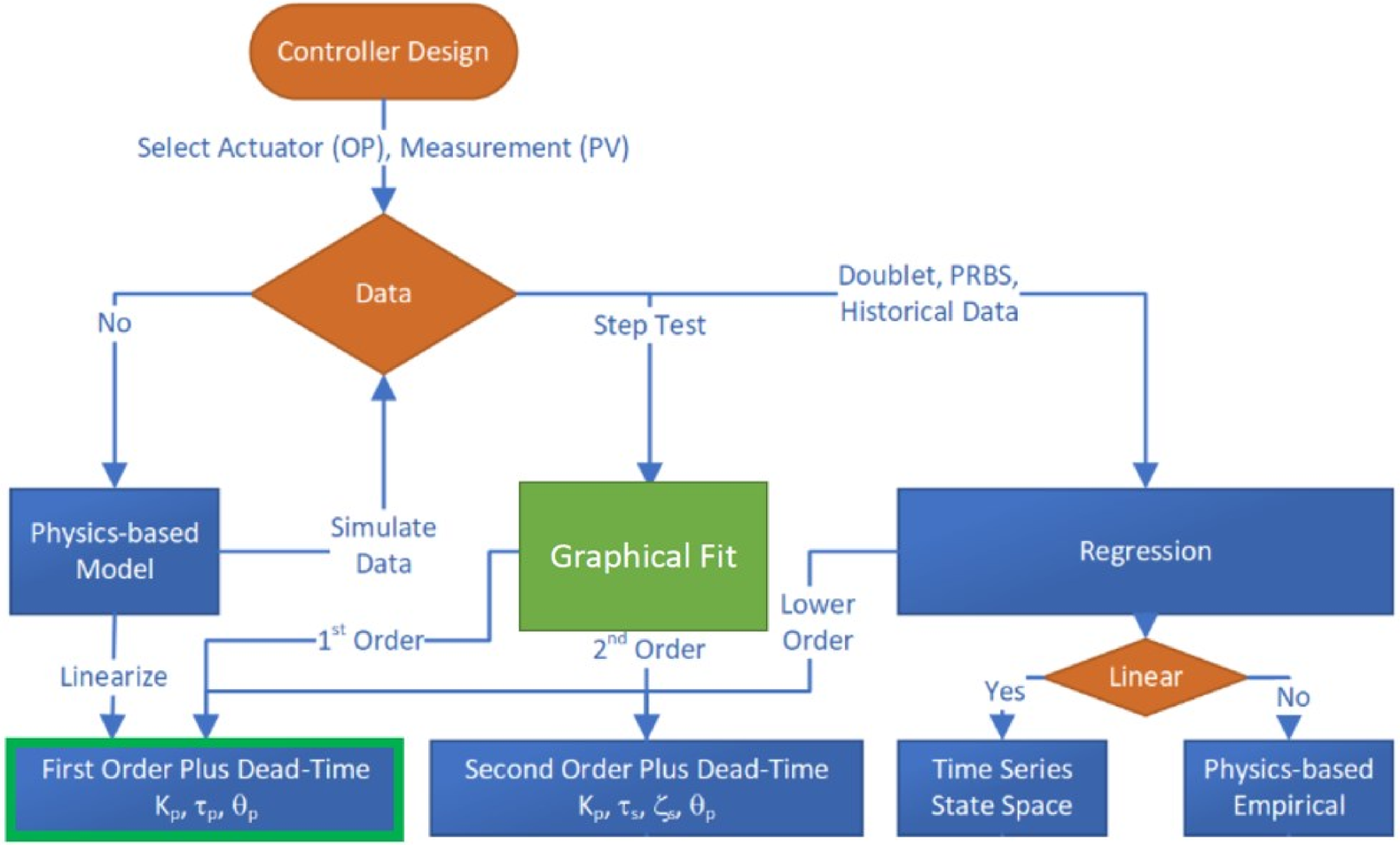


Kp	Ti(s)	Td(s)
13.10	43.36	6.58

Overshoot (%)	SS Error (%)	Rising Time (s)	IAE	ITAE
1.56	0.47	63.74	$2.753e+03$	$0.112e+06$

3.5 การทดลองควบคุมอุณภูมิ

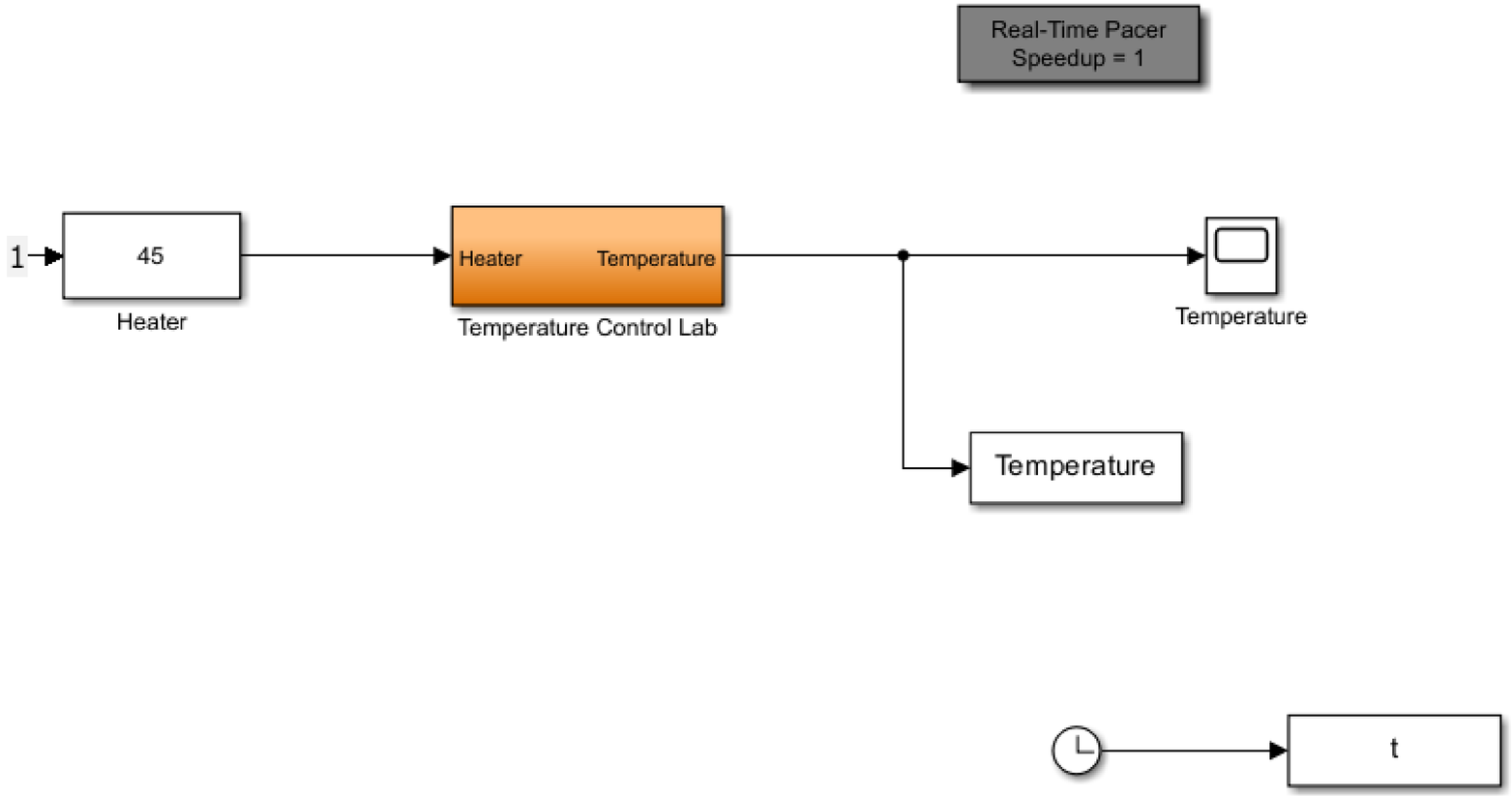
- First Order Plus Dead Time Models



https://apmonitor.com/pdc/index.php/Main/HomePage

3.5 การทดลองควบคุมอุณภูมิ

- First Order Plus Dead Time Models



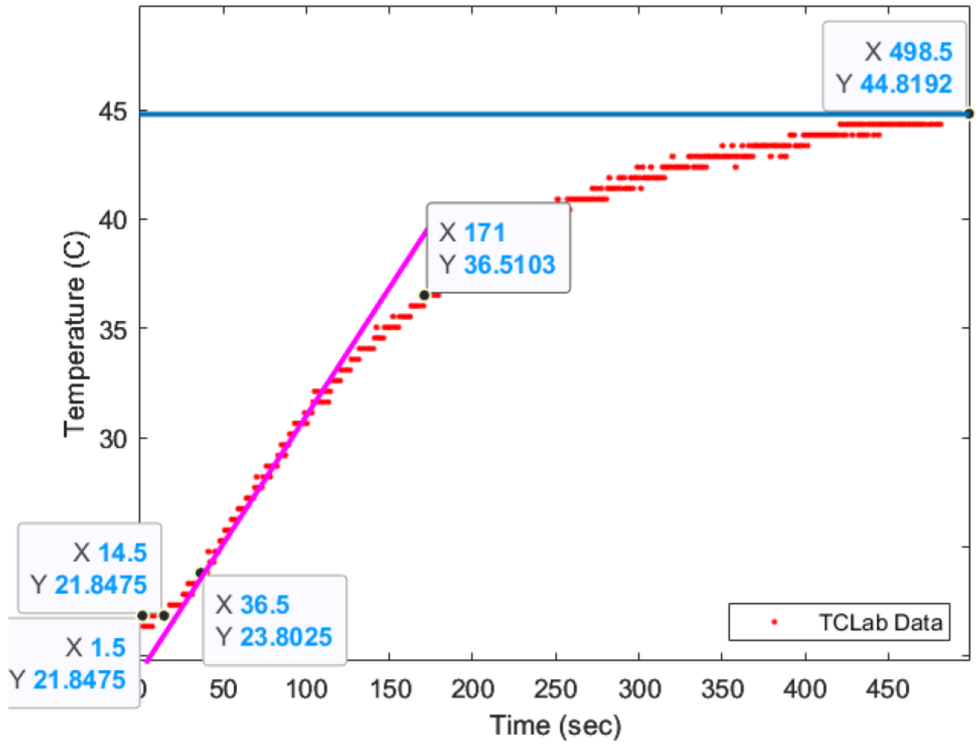
3.5 การทดลองควบคุมอุณภูมิ

- First Order Plus Dead Time Models

Temperature response to a step input and obtain the First Order Plus Dead Time FOPDT parameters

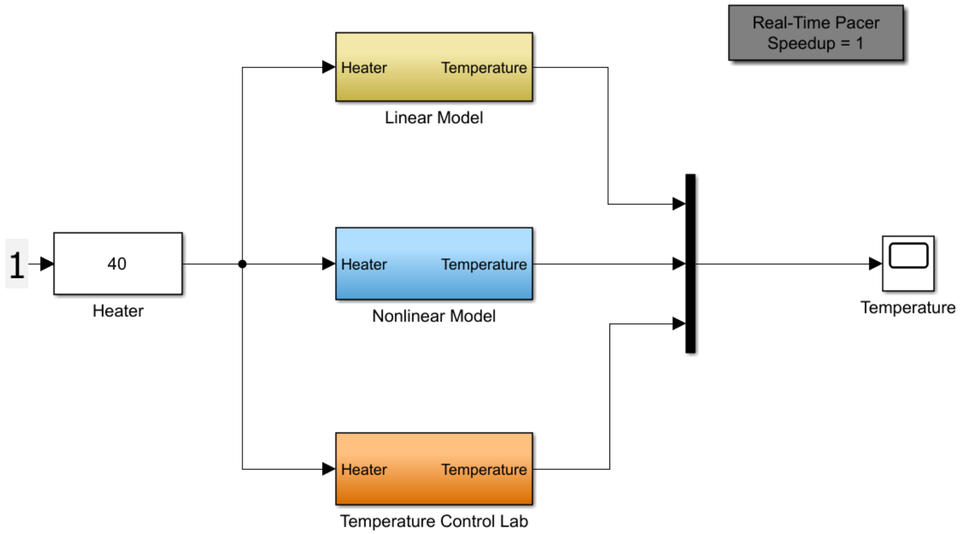
dead_time	13
du	90
dy	45.7683
Kp	0.5085
t	*1001x1 doub*
Tao	123.2000
Temperature	*1001x1 doub*
tout	*1001x1 doub*
y_tao	51.9256

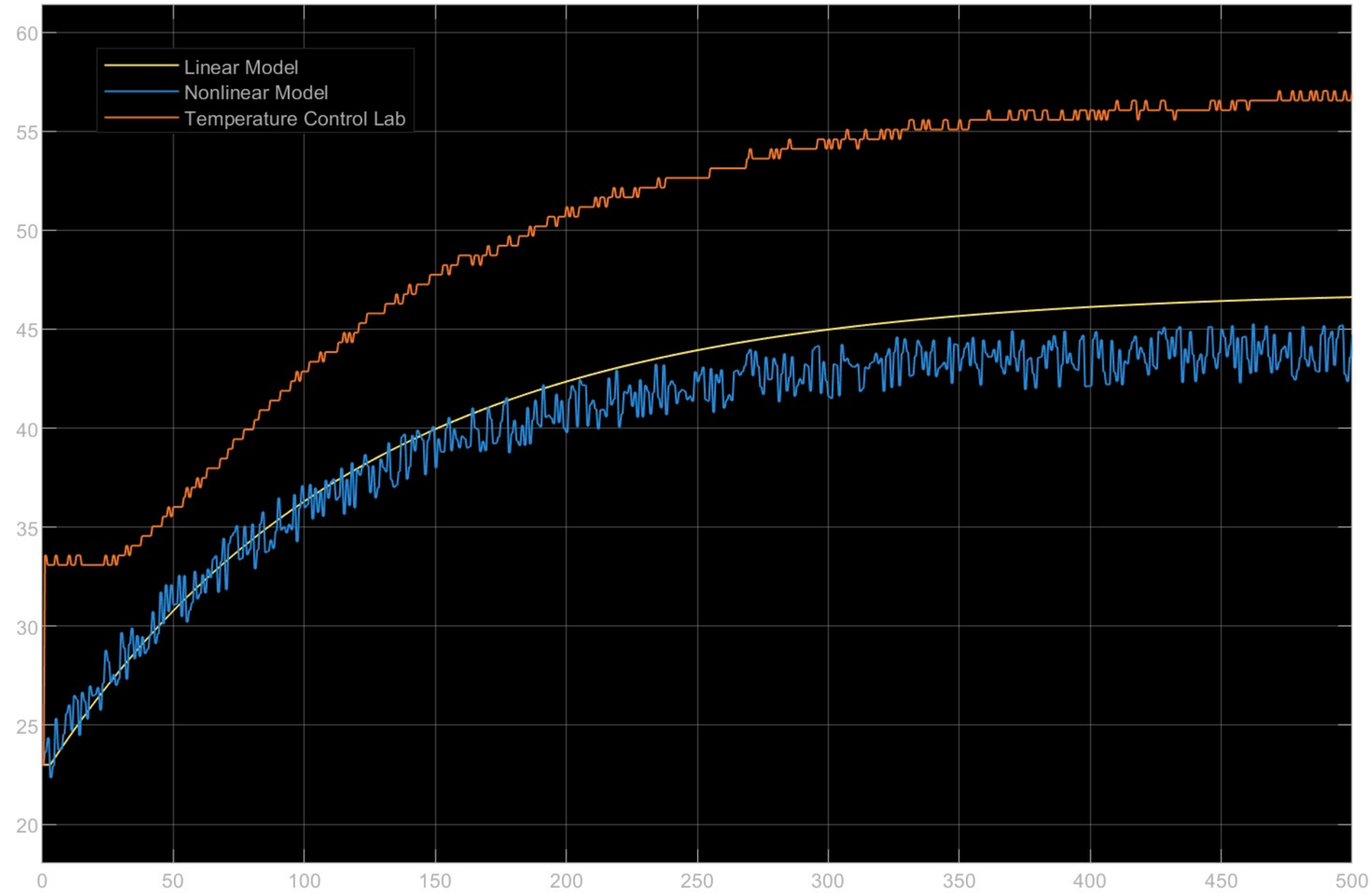
```
dy=Temperature(end)-Temperature(1);
du=45;
Kp=dy/du;
dead_time=22-9;
y_tao=Temperature(1)+0.632*dy;
Tao=145.2-22;
```



3.5 การทดลองควบคุมอุณภูมิ

- First Order Plus Dead Time Models





3.5 การทดลองควบคุมอุณภูมิ

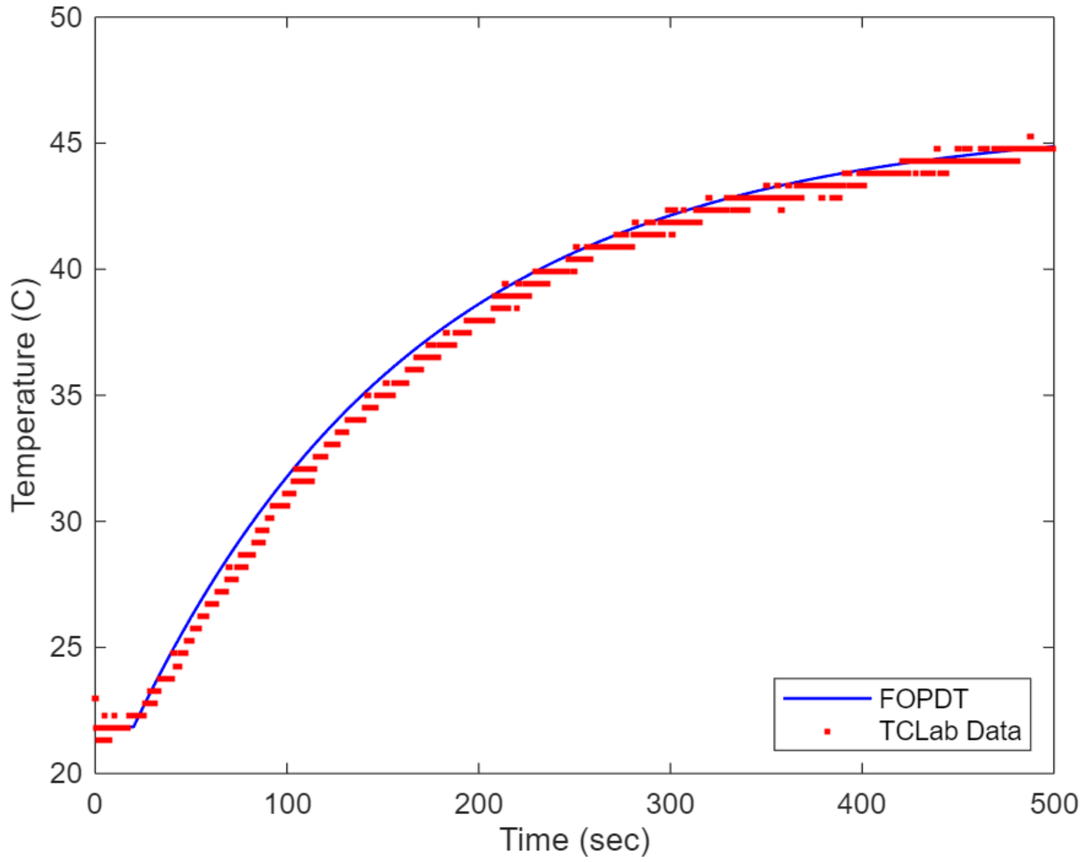
- First Order Plus Dead Time Models

FOPDT fitting

```
clc; clear all;
load ('data_TClab_fit.mat')

clf
figure(2)
K2 =   32                ;
Tau2 =  150              ;
Theta2 =    20           ;

n = 500;
time = linspace(1, n, n);
delay = ones(1, n);
for i=1:n
    if time(i)<Theta2
        delay(i) = 0;
    end
end
y = Temperature(4) + .75 * K2 * delay .* (1.0-exp(-(time-Theta2)/Tau2));
plot(time, y, 'b-', t,Temperature,'r.', "linewidth", 1)
legend('FOPDT', 'TCLab Data', 'location', 'southeast')
xlabel('Time (sec)')
ylabel('Temperature (C)')
```



3.5 การทดลองควบคุมอุณภูมิ

SCHOOL OF ENGINEERING KMITL

- First Order Plus Dead Time Models

A first-order linear system with time delay is a common empirical description of many stable dynamic processes.

Graphical Method: FOPDT to Step Test

After gaining an intuitive understanding of the step response of a single heater model, it is important to understand the mathematical FOPDT equation

$$\tau_p \frac{dy(t)}{dt} = -y(t) + K_p u(t - \theta_p)$$

with variables $y(t)$ and $u(t)$

Where

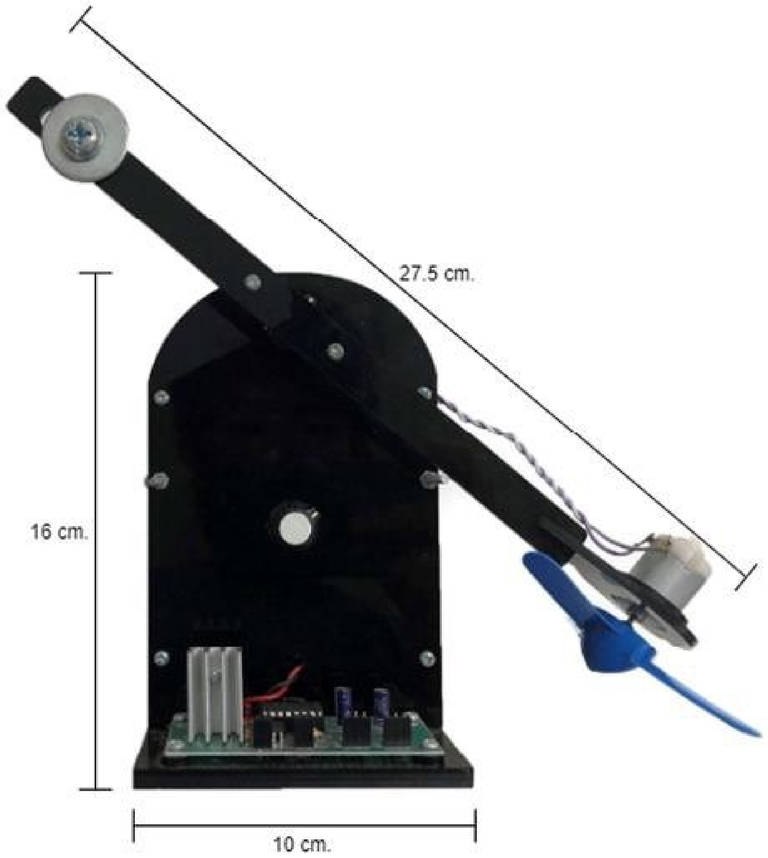
K_p = Process gain

τ_p = Process time constant

θ_p = Process dead time

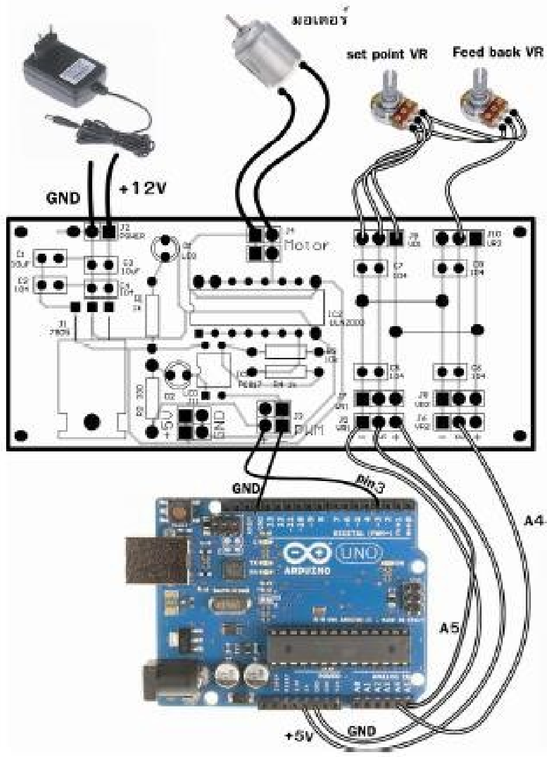
3.6 การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)

- ระบบบินขึ้นและลงจอด (VTOL)



3.6 การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)

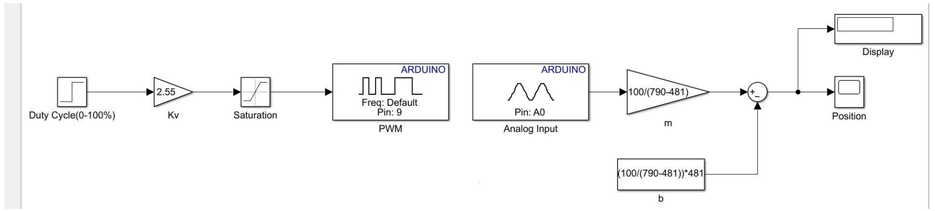
- Wiring

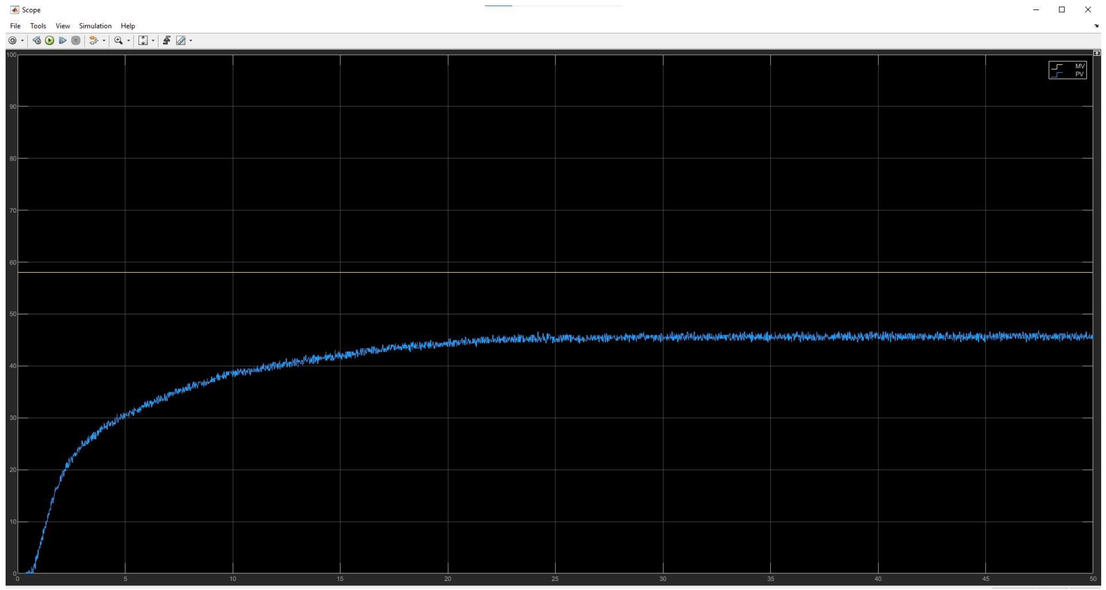


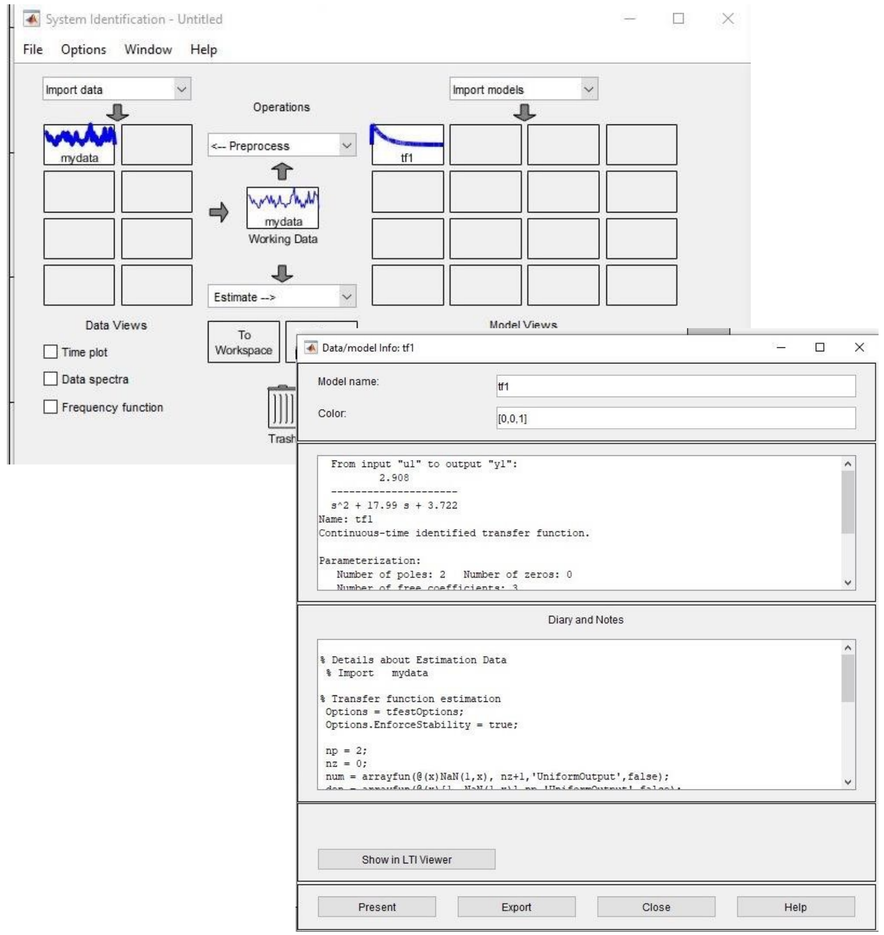
เครื่องมือและอุปกรณ์	จำนวน
1. Vertical take-off and landing	1
2. บอร์ด Arduino Uno R3 พร้อม USB	1
3. สาย Jumper	8
4. คอมพิวเตอร์	1

3.6 การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)

- โมเดลทางคณิตศาสตร์ (System Identification Toolbox)

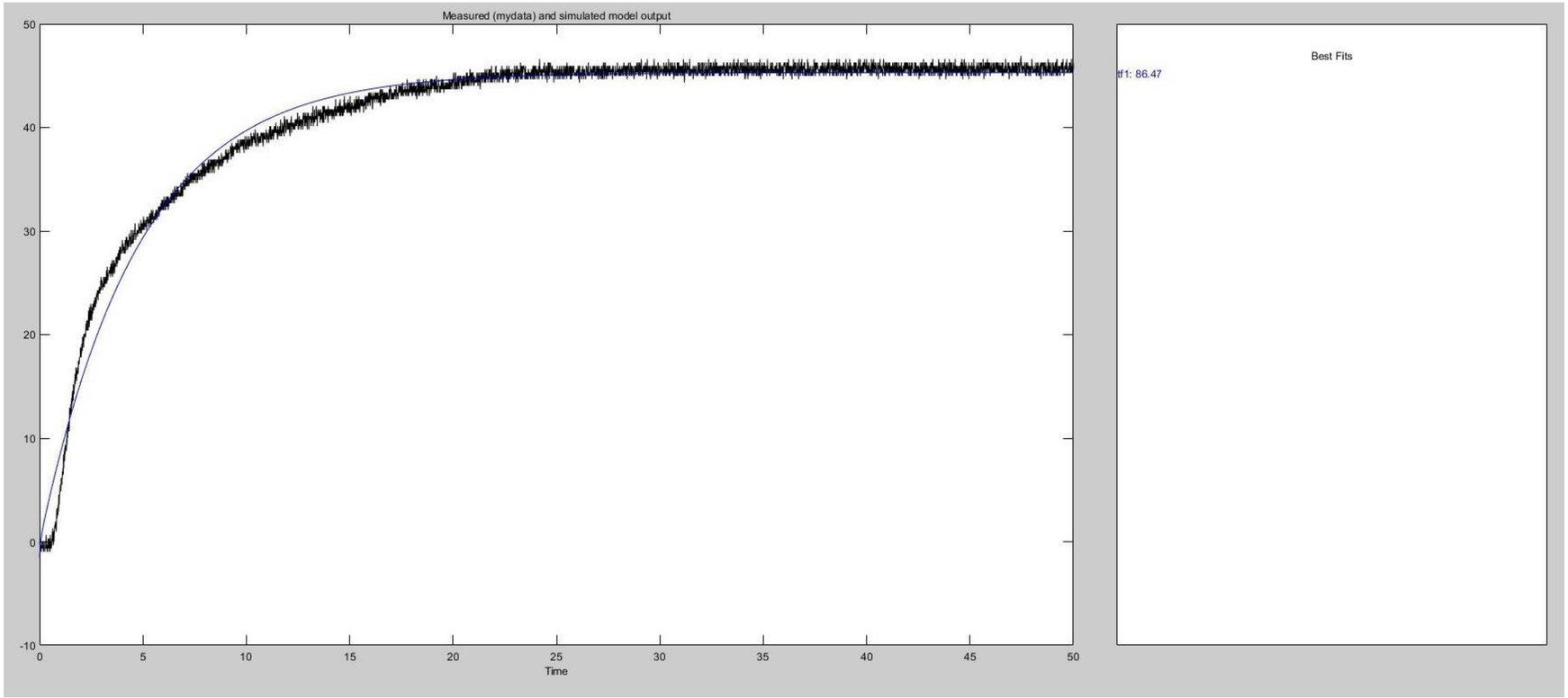






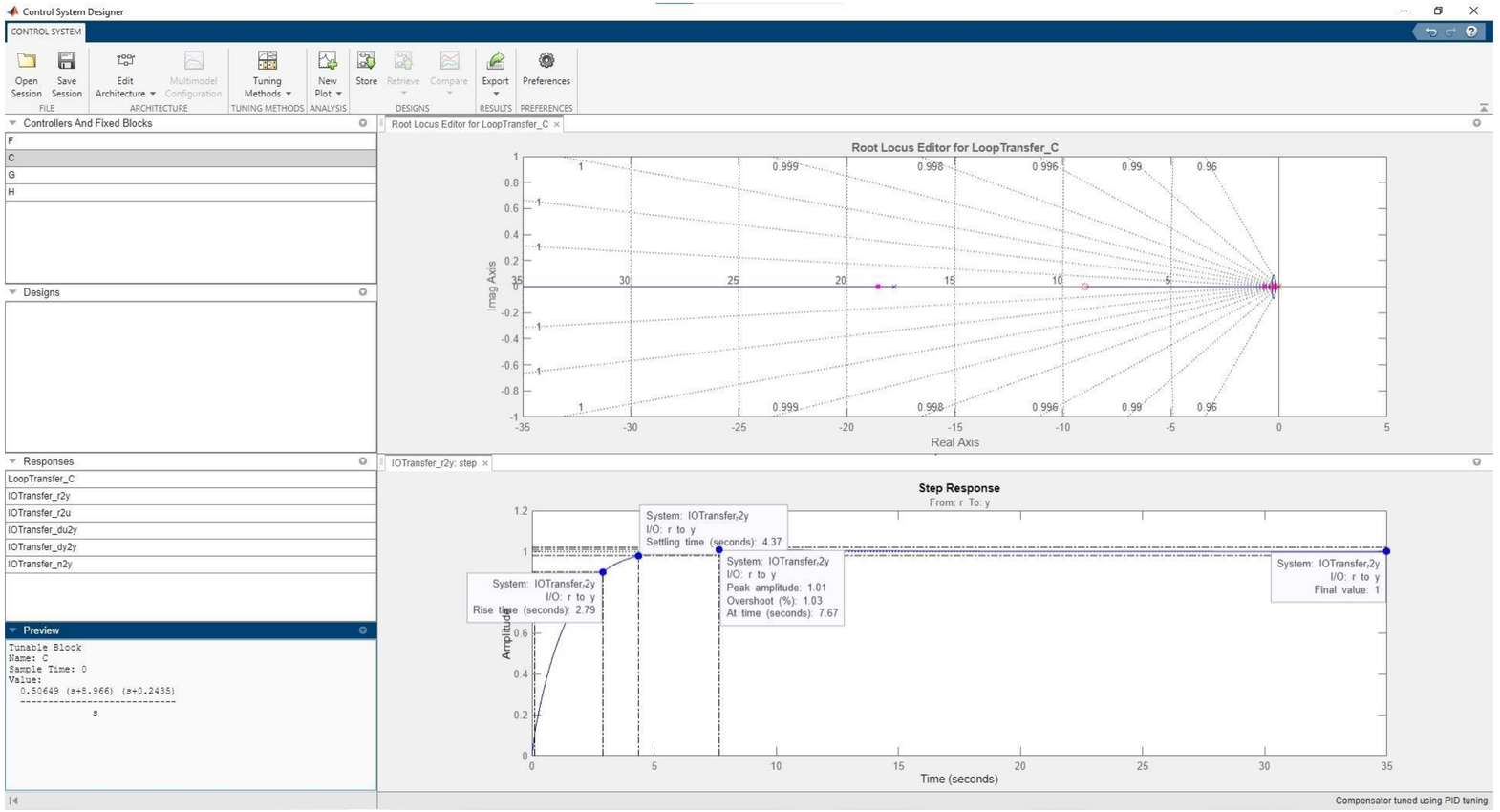
3.6 การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)

- โมเดลทางคณิตศาสตร์ (System Identification Toolbox)



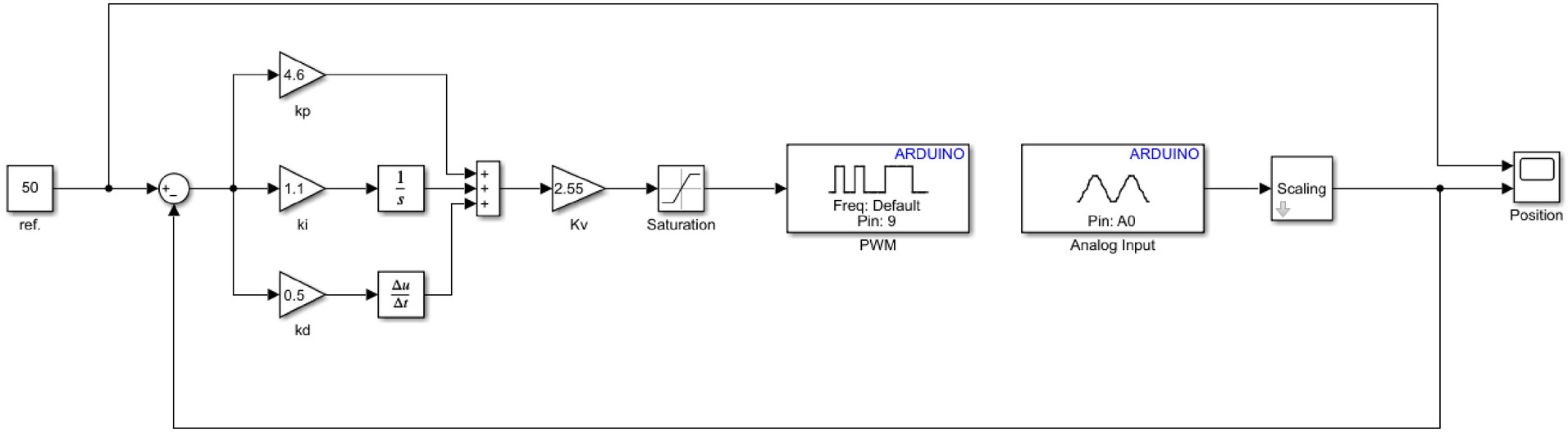
3.6 การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)

- SISO Tool Design (Root Locus)



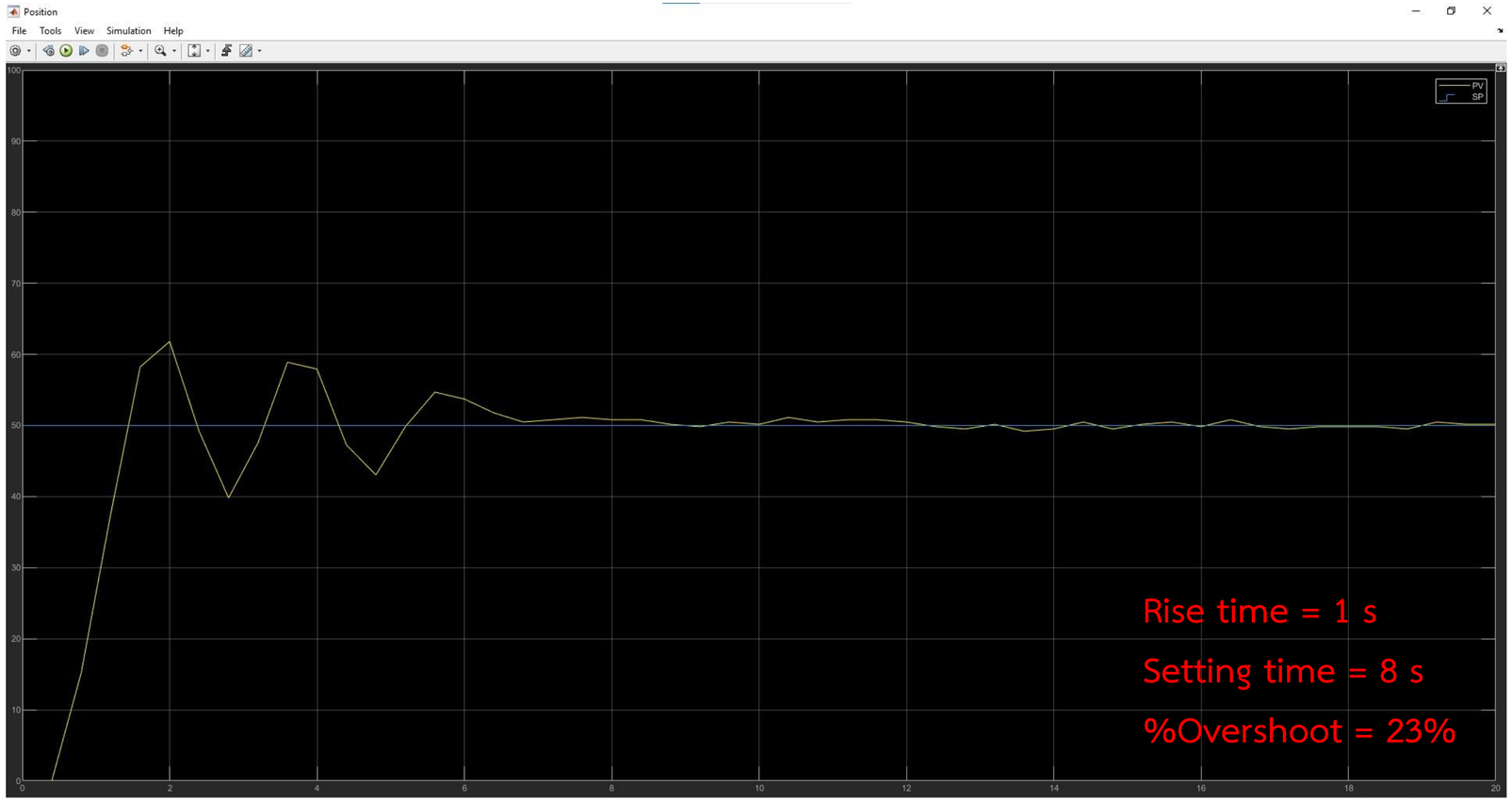
3.6 การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)

- Hardware-in-the-loop



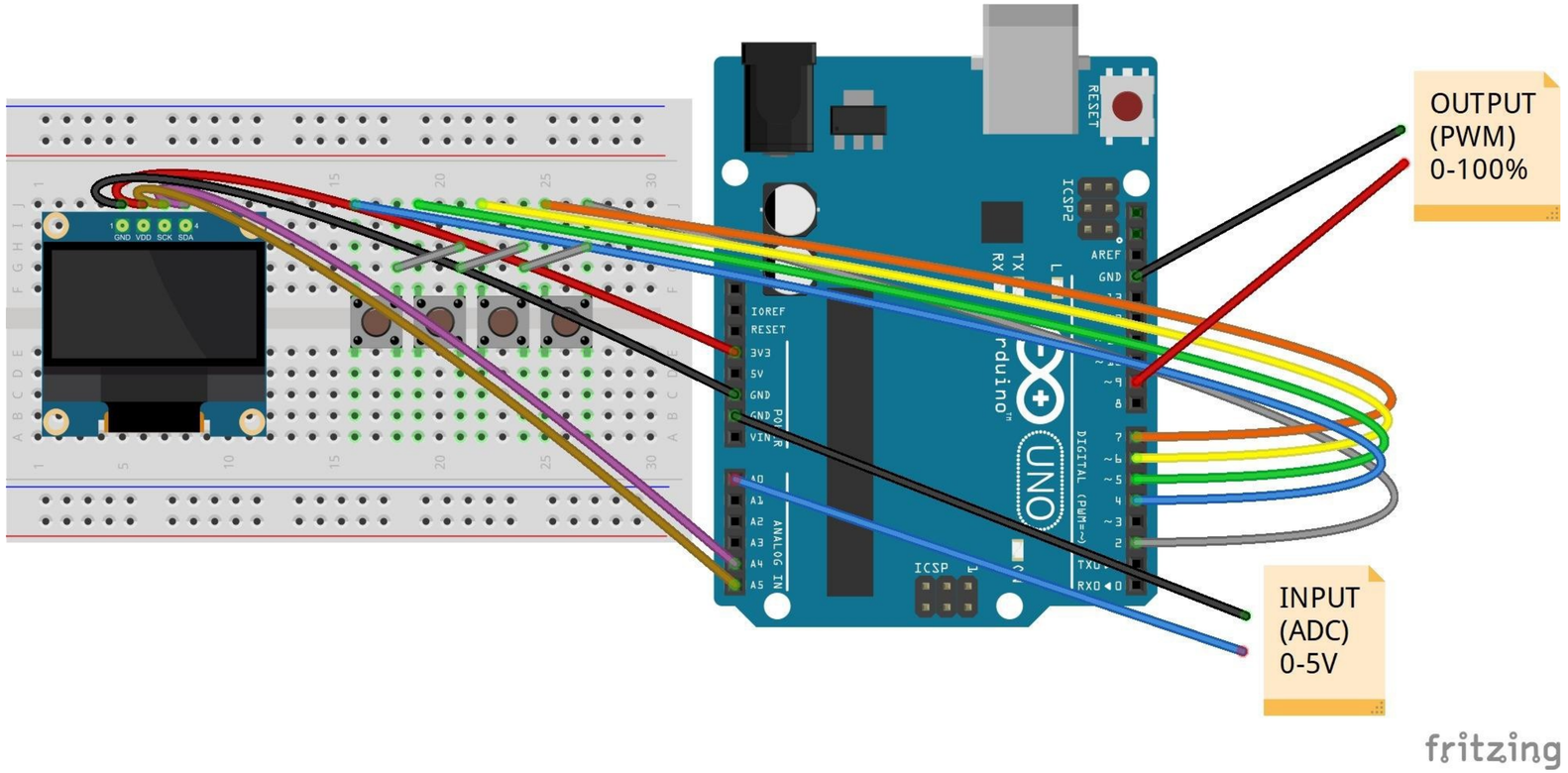
3.6 การทดลองควบคุมระบบบินขึ้นและลงจอดแนวดิ่ง (VTOL)

- Response



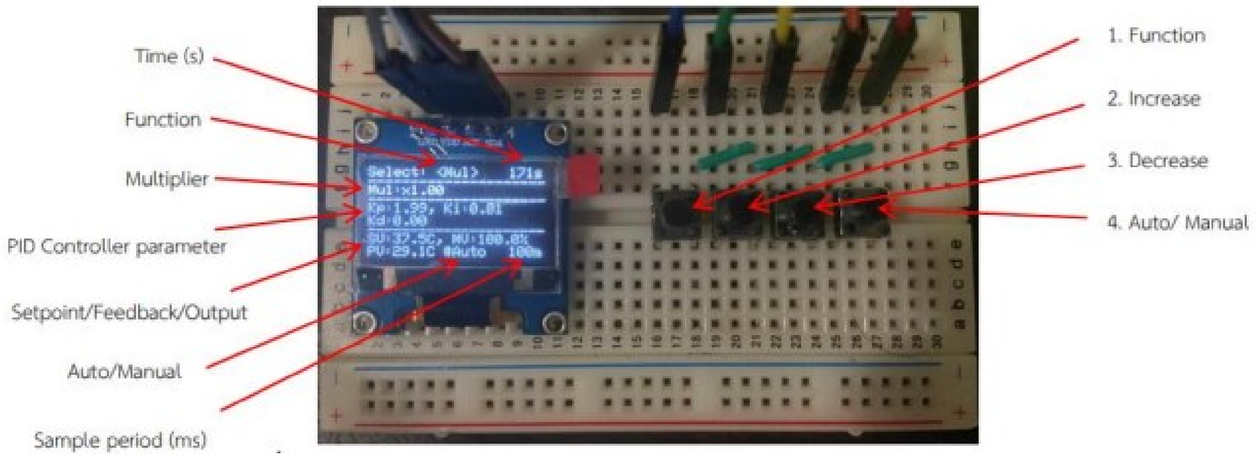
3.7 PID Stand-Alone with Arduino Uno R3

- Wiring



3.7 PID Stand-Alone with Arduino Uno R3

- PID tuning



ฟังก์ชันการใช้งานของตัวควบคุม PID (Arduino)

1. ส่วนควบคุมประกอบด้วย 4 ปุ่มคือ

ปุ่มที่ 1 คือ กดเพื่อเลือกฟังก์ชันการใช้งานจะประกอบด้วย

<Mul> คือเลือกจำนวนเท่าของตัวคูณ 0.01, 0.1, 1, 10 และ 100 เท่า

<Kp> คือเลือกการปรับค่า Kp (0.00 -100.00)

<Ki> คือเลือกการปรับค่า Ki (0.00 -100.00)

<Kd> คือเลือกการปรับค่า Kd (0.00 -100.00)

<SV> คือเลือกการปรับค่าเป้าหมาย (0.00 -100.00)

<MV> คือเลือกการปรับค่าสัญญาณควบคุม (0.00 -100.00)

ปุ่มที่ 2 คือ ปุ่มเพิ่มค่าตามจำนวนเท่าของตัวคูณที่เลือกก่อนหน้า

ปุ่มที่ 3 คือ ปุ่มลดค่าตามจำนวนเท่าของตัวคูณที่เลือกก่อนหน้า

ปุ่มที่ 4 คือ ปุ่มเปลี่ยนโหลดการทำงานแบบ Auto หรือ Manual

2. ส่วนแสดงผลประกอบด้วยการแสดงผลดังนี้

Time (s) คือ เวลาตลอดการควบคุม

Function คือสถานะล่าสุดของฟังก์ชันต่าง ๆ (Mul, Kp, Ki, Kd, SV และ MV) เพื่อปรับค่า

Multiplier คือตัวคูณ (0.01, 0.1, 1, 10 และ 100)

PID Controller parameter คือ Kp, Ki และ Kd

Setpoint/Feedback/Output คือค่า SV, PV และ MV

Auto/Manual คือโหมดการควบคุม

Sample Time (ms) คือ จำนวนเวลาของวงรอบการควบคุม PID

3.8 คำถามท้ายการทดลอง

SCHOOL OF ENGINEERING KMITL

- จงอธิบายการประยุกต์ใช้ MATLAB AppDesigner สำหรับการวิเคราะห์และออกแบบระบบควบคุมที่สนใจ
- จงยกตัวอย่างการบวนการ FO, SO, FODPDT, IPDT, Unstable
- จงอธิบายความแตกต่างของ SIL, HIL, PIL, Stand-alone
- จงยกตัวอย่างตัวควบคุม PID ที่ใช้กับกระบวนการในอุตสาหกรรม

INSTRUMENTATION ENGINEERING LABORATORY 2

0106810 2

SCHOOL OF ENGINEERING KMITL